

Password-Authenticated Key Exchange Protocols: A Survey

DING WANG, GUANLING LI, and JINGWEI JIANG, Nankai University

Password-authenticated key exchange (PAKE) protocols tackle the important problem of how to enable two parties, who share a low-entropy password, to establish a cryptographically strong session key for secure data communication. Although considerable research efforts have been devoted to designing hundreds of PAKE protocols, to the best of our knowledge, there have been few systematic reviews. In this work, we provide a comprehensive overview of PAKE research. We first propose a list of 13 desirable properties of PAKE protocols in terms of security and usability, enabling PAKE protocols to be systematically rated across a common spectrum. We then provide a taxonomy for PAKE protocols, and classify them into seven types according to their underlying design strategies. For each type, we investigate the inner working mechanisms of various representative protocols, and identify their pros, and cons. We further classify existing PAKE protocols from five other key perspectives (i.e., symmetry, number of participants, hardness assumptions, security goals, and round complexity) and review their development history under each classification, aiming to provide an in-depth and thorough understanding of the status-quo of PAKE research. Based on 13 properties and six perspectives, we conduct a large-scale comparative evaluation of 71 representative PAKE protocols in a systematic manner. Finally, we highlight a few potential directions for the future design of PAKE protocols.

CCS Concepts: • **Security and privacy** → **Authentication**; • **Theory of computation** → **Cryptographic protocols**; • **General and reference** → **Surveys and overviews**.

Additional Key Words and Phrases: Password-authenticated key exchange, design strategy, evaluation criteria

ACM Reference Format:

Ding Wang, Guanling Li, and Jingwei Jiang. 2025. Password-Authenticated Key Exchange Protocols: A Survey. *ACM Comput. Surv.* 55, 12, Article 1 (November 2025), 37 pages. <https://doi.org/10.1145/xxx>

1 INTRODUCTION

No matter for traditional user-server environments or cloud/edge computing environments, user authentication constitutes one of the most critical lines of defense, ensuring that the communicating parties are intended entities. Existing methods for user authentication fall into three broad categories: something the user knows (e.g., passwords), something the user has (e.g., smart cards, USB sticks), and something the user is (e.g., fingerprint, iris, and facial images) [1]. Among them, password-based authentication requires users only to remember and type a (username, password) pair. Thus, the user incurs almost no cost and can move conveniently without carrying hardware tokens. The latter two authentication methods may provide higher security or usability, but do poorly in deployability [2]. In fact, due to the advantages of simplicity and convenience, none of these alternatives proposed in recent years can replace passwords [2–4], and passwords are likely to remain the dominant user authentication method in the foreseeable future [5, 6].

This work was supported in part by the National Natural Science Foundation of China under Grant 62222208, and by the Fundamental Research Funds for the Central Universities (Nankai University) under Grant No. 63253229.

Authors' addresses: Ding Wang, Guanling Li, and Jingwei Jiang are with the College of Cryptology and Cyber Science, Nankai University, Tianjin 300350, China, and also with the Key Laboratory of Data and Intelligent System Security (Nankai University), Ministry of Education, Tianjin 300350, China, email: wangding@nankai.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Association for Computing Machinery.

0360-0300/2025/11-ART1 \$15.00

<https://doi.org/10.1145/xxx>

of this paper focuses on a similar design strategy reused in a family of protocols (see Fig. 2) and results in seven types, because a password may play multiple roles in a PAKE protocol, confusion may occur when classifying PAKE protocols based on how the password is used. For example, in Jarecki et al.'s OPAQUE protocol [7], the password pw is used to derive the generator (i.e., $rw = H(pw, H'(pw)^k)$), and the mapping of the password (i.e., rw) is also used as an encryption key (i.e., $c = \text{Enc}_{rw}(a, A, B)$), where $H(\cdot)$ and $H'(\cdot)$ are hash functions, $\text{Enc}(\cdot)$ is encryption function, k is the server's key, and (a, A, B) are the server's long-term HMQV keys. According to the way of using passwords, the OPAQUE protocol belongs to type-1 and 2 in [22]. But from our perspective of the similar design strategy, OPAQUE will be classified into only one type (see Tables 3).

Second, we propose a set of criteria with 13 desirable properties covering various security and usability goals, and conduct a large-scale comparative evaluation, which is much more comprehensive than that of Hao-Oorschot's work [22]. It is known that it is incredibly difficult to design a PAKE protocol correctly. For example, the seminal EKE [13] and SRP [10] are found to be vulnerable to undetectable online guessing attacks [21, 23]; SPEKE [15] is pointed out to be vulnerable to impersonation attacks and key-malleability attacks in 2018 [20], twelve years after it was standardized by ISO/IEEE. Moreover, there are various properties (e.g., perfect forward secrecy, user anonymity) desirable for varied application environments, but protocol designers often only illustrate the superior aspects over previous protocols (see Sec. 4.4). All this implies that an objective and comprehensive evaluation criterion set for PAKE protocols is necessary.

Third, besides the design strategy, we explore PAKE protocols from five other perspectives. Ideally, one can classify PAKE protocols according to a unified method to facilitate comparative analysis. Unfortunately, to solve practical problems in different applications, PAKE protocols are usually proposed with special structures, such as multi-party [24, 25], anonymous [26, 27], and quantum-resistance [28, 29]. The large number and complex structure of PAKE protocols hinder the systematic analysis. It is virtually impossible to make a thorough comparison of all PAKE protocols based on one method alone. Using multiple classification methods will better illustrate the differences, pros, and cons of existing protocols. In order to supplement the information not covered by the design strategy, we understand the development of PAKE from five perspectives: symmetry, number of participants, hardness assumptions, security goals, and round complexity.

Fourth, we provide an overview of ten PAKE protocols using smooth projective hash functions (SPHF), which is not by Hao-Oorschot [22]. The security of PAKE protocols using SPHF usually relies on the lattice, e.g., [30–32]. The analysis of such protocols not only enhances the completeness of the PAKE survey but also is helpful for guiding the design of quantum-secure PAKE protocols. Further, we explore newly proposed PAKE protocols (e.g., KHAPE [11] and aEKE-HMQV [8]), which perform well in terms of security, computation overhead, or round complexity (see Fig. 3).

Scope. To keep the scope of the article manageable, we consider works that focus on the PAKE protocol, stemming from the pioneering PAKE protocol in 1992 [13]. It should be noted that, as hundreds of PAKE protocols have been proposed, this survey mainly focuses on PAKE protocols that are highly representative or clarify a certain mainstream technique route (see Fig. 2). Due to page limits, our work is not intended to be absolutely complete. Some subfields of PAKE (e.g., threshold PAKE and PAKE based on a chaotic map) are not discussed. We use Google Scholar and the database Web of Science as the main tools to discover related papers. In addition, we screen most of the related high-profile conferences, such as EUROCRYPT, CRYPTO, ASIACRYPT, ACM CCS, USENIX SEC, IEEE S&P, and NDSS, just to name a few, to find out recent works. The major keywords we have used include “password-authenticated key exchange”, “password-based authentication key exchange”, “password-only authentication and key exchange”, and “PAKE”.

Contributions. This paper provides a systematic overview of password-authenticated key exchange protocols. In all, our contributions are summarized as follows:

- We provide a systematic framework for evaluating PAKE protocols, including a security model and a criterion set of 13 desirable properties in terms of security and usability.
- We conduct a comprehensive review of PAKE protocols, and for the first time classify them into seven types according to their underlying design strategies.
- We revisit existing PAKE protocols in terms of five other perspectives (i.e., symmetry, number of participants, hardness assumptions, security goals, and round complexity) and discuss representative protocols of each classification to clarify their working mechanisms.
- We conduct a large-scale comparative evaluation of 71 representative PAKE protocols in a systematic manner based on our evaluation framework and six perspectives. We also provide a few insights into promising future research directions of the PAKE protocol.

2 PRELIMINARIES

In this section, we summarize the phases for designing PAKE protocols, and identify the security model and the desirable properties in terms of security and usability.

2.1 Protocol Phases

To date, an array of PAKE protocols with different structures (e.g., symmetric [16], asymmetric [10], and multi-party [33]) have been published in the literature. Even though the structure may be complicated, the fundamental principle of PAKE protocols is to achieve authentication using only a low-entropy password. Hence, the phases of most PAKEs are largely the same: 1) The setup phase, which initializes the system's public parameters; 2) The registration phase, in which user identities and passwords are established; and 3) The login phase, which verifies the legitimacy of users.

- *Setup.* Setup is a preparatory phase where the server selects and publishes a set of necessary parameters for subsequent interaction with users. For example, the PAK protocol [34] publishes a multiplicative group $\mathbb{G}$ with generator g and prime order p , and the PEKEP protocol [35] publishes an RSA public key pair (N, e) . In addition, some PAKE protocols (e.g., PFS-SPAKE2 [14]) collectively refer to the setup and registration phases as initialization. But in this article, we only regard the computation without passwords (i.e., setup) as initialization.
- *Registration.* The registration phase is assumed to be executed over a channel that is secure against network adversaries (e.g., face-to-face, under TLS protection). Although some articles (e.g., [36]) describe the login phase in detail and omit the registration phase, the registration phase is an essential process. More specifically, a user chooses their identity (also called username) and password according to the server's policy (e.g., the username is different from existing ones, and the password length is greater than 8). Then the user sends a (username, password) pair to the server, and the server may store passwords in different ways. For example, A-EKE [37] stores a hash of the password; AMP [38] stores a salted hash of the password and a random salt; OPAQUE [7] generates the password file with HMQV [39].
- *Login.* It is also called the login and authentication phase. In a PAKE protocol, both the user and the server prove their identity through a password: The user can type the correct password, and the server stores the password (or its verifier). This is different from the password-over-TLS scheme. Because in password-over-TLS, the user authenticates itself with a password, but the server authenticates itself with a certificate. It means that PAKE can achieve authentication between two parties without a public key infrastructure. Moreover, protocol designers employ various cryptographic primitives (e.g., oblivious pseudo-random functions [7], commitments [40], and public key juggling [16]) to design PAKE protocols.

Table 1. Capabilities of the adversary.

C-1	The adversary can offline enumerate the Cartesian product $D_{id} \times D_{pw}$, where D_{id} and D_{pw} denote the identity space and the password space, respectively.
C-2	The adversary can determine the victim's identity.
C-3	The adversary has full control of the communication channel.
C-4	The adversary can learn the victim's password.
C-5	The adversary can learn the previous session key(s).
C-6	The adversary can learn the server's long-term private key(s), as well as all other data stored on the server, but only when evaluating an eventual server compromise (e.g., forward secrecy).

2.2 Security Model

Generally, it is difficult for authors to make a comprehensive evaluation of their protocols. On the one hand, authors may unintentionally ignore the shortcomings of their protocols (e.g., [13, 15, 36]), and give high ratings to certain properties that their protocol meets (e.g., the anonymity [27, 41, 42]). On the other hand, it is challenging to clearly evaluate PAKE protocols by considering multiple factors, such as differences in application scenarios, assumptions on the adversary's capabilities, and requirements for the security goals (see Sec. 4). Hence, a rigorous security model that can simulate the practical protocol execution is necessary.

To simulate PAKE protocols more realistically, we need to clarify the capabilities of an adversary before defining the security model. Our adversary model is based on Bellare-Pointcheval-Rogaway (BPR) model [43] and other models, particularly those presented in [44–48]. The BPR security model is widely used in the PAKE literature (e.g., [29, 32, 49–51]). We summarize the capacities of the adversary $\mathcal{A}$ for PAKE protocols in Table 1. Each capability is realistic and reasonable.

Specifically, the capability C-1 in Table 1 comes from the vulnerable behaviors of selecting passwords. Boneau's work [52] indicates $|D_{id}| \leq |D_{pw}| \approx 10^6$. $\mathcal{A}$ can offline enumerate the Cartesian product $D_{id} \times D_{pw}$ in polynomial time. The capabilities of C-2 and C-3 are from the characteristics of network communication. In a non-anonymous network, $\mathcal{A}$ can easily obtain the source address, the destination address, and the user's identity through IP packets [53]. In addition, an attacker can eavesdrop, insert, delete, and modify any transmitted messages over the public channel by using network packet analysis tools. For the capability of C-4, if the password-entry interface is accidentally infected with viruses [54], an attacker can intercept the password entered by the user. The capability of C-5 may be due to the improper erasure of session keys after use, or the compromise of a host computer [44, 46]. Moreover, for the capability of C-6, when an attacker breaks into a computer system, the password file stored on the server side may be leaked (see [55]).

The BPR model [43] simulates the capabilities of an adversary by querying oracles. The security model presented here is applied to the basic client-server setting, which can be easily extended to other settings, such as the two-server setting and wireless sensor network.

Participants. We assume a user (also called client) $U \in \mathcal{U}$ and a server $S \in \mathcal{S}$, which participate in the password-authenticated key exchange (PAKE) protocol $\mathcal{P}$. Note that $\mathcal{U}$ and $\mathcal{S}$ are disjoint, i.e., $\mathcal{U} \cap \mathcal{S} = \emptyset$. We denote client instances by $U^i, i \in \mathbb{Z}$ and server instances by $S^j, j \in \mathbb{Z}$. In addition, we denote any kind of instance by $I \in \mathcal{U} \cup \mathcal{S}$.

Password and long-term keys. Each user U holds an identifier ID_U and a password PW_U . We assume that each PW_U is from a dictionary $\mathcal{D}$ that obeys the Zipf's law [48]. The size $|\mathcal{D}|$ is independent of the security parameter. Each server S stores a long-term key x and a password file (e.g., ID_U and a hash of the password PW_U in [37]).

Queries. The adversary $\mathcal{A}$ is a probabilistic polynomial time (PPT) algorithm with capabilities defined in Table 1. $\mathcal{A}$ has access to the following oracles:

- $\text{Execute}(U^i, S^j)$: This query models the ability of $\mathcal{A}$ to perform passive attacks. That is, $\mathcal{A}$ eavesdrops on honest executions between U^i and S^j . This query returns the protocol transcript (e.g., the complete ordered messages exchanged) to $\mathcal{A}$.
- $\text{Send}(I, m)$: This query models the ability of $\mathcal{A}$ to perform active attacks. $\mathcal{A}$ sends a message m to the instance I . After receiving m , I runs according to the protocol specification and updates its state. Finally, the message output by I is given to $\mathcal{A}$.
- $\text{Test}(I)$: This query defines the session key's semantic security. If the instance I has computed a session key sk , then a private coin c is flipped. If $c = 0$, it returns a random key of the same size chosen uniformly at random. If $c = 1$, it returns the session key sk of the instance I . If no session key is defined for instance I , it returns the undefined symbol $\perp$.
- $\text{Reveal}(I)$: This query models the corruption of session keys. If the instance I has computed a session key sk , and neither I nor its partner was asked by a Test query, it returns the current session key sk of the instance I . Otherwise, it returns $\perp$.
- $\text{Corrupt}(I)$: This query models the corruption of the instance I . If $I = U$, it outputs the password PW_U of U ; if $I = S$, it outputs the secret key and the password file of the server S .

Accepting. If an instance I has run the protocol successfully and computed a session key sk , the instance I is considered to be accepted. Otherwise, it is considered to be rejected.

Partnering. We denote the session identifier by sid , which may be a concatenation of all messages sent and received by the instance [30, 31, 56]. Instances U^i and S^j are partnered if:

- Both instances have been accepted, i.e., $sk_U^i = sk_S^j$;
- Both instances shared the same session identifier, i.e., $sid_U^i = sid_S^j$;
- The partner identifier of U^i is $pid_U^i = S$, and the partner identifier of S^j is $pid_S^j = U$.

Freshness. The notion of freshness captures the fact that a session key can't be trivially known to the adversary $\mathcal{A}$. An instance I is considered to be fresh if:

- I has accepted a session key sk ;
- Neither I nor its partner has experienced the Reveal -query or Corrupt -query.

Security. The security notion requires that an external adversary $\mathcal{A}$ cannot distinguish the real session key from a random string of the same size in polynomial time. $\mathcal{A}$ breaks the security of the protocol $\mathcal{P}$ if it makes a Test query to a fresh instance, and outputs a bit c' with $c' = c$. We define this event by Succ . The advantage of $\mathcal{A}$ in breaking this game is

$$\text{Adv}_{\mathcal{P}}^{\text{ake}}(n)(\mathcal{A}) = 2\Pr[\text{Succ}(\mathcal{A})] - 1 = 2\Pr[c' = c] - 1. \quad (1)$$

We say that a PAKE protocol $\mathcal{P}$ is semantically secure if for all dictionaries $\mathcal{D}$ and any PPT adversary $\mathcal{A}$ making at most q_{send} online guessing attacks, $\text{Adv}_{\mathcal{P}}^{\text{ake}}(\mathcal{A})$ is negligible after excluding the advantages of online password guessing.

2.3 Evaluation Criteria

At IEEE S&P'12, Boneau et al. [2] made a comparative evaluation of 35 representative web authentication schemes (including textual passwords, hardware tokens, and biometrics) based on 25 criteria (properties), and pointed out the irreplaceability of passwords. In the field of PAKE, some desirable properties have well attracted the attention of researchers, such as forward secrecy [14, 57], user anonymity [27, 42], resistance to pre-computation attacks [7, 58], and round-optimal [30, 32], while others are not (e.g., resistance to ephemeral secret leakage attacks, resistance to known-key attacks). Building on the efforts of previous works (e.g., [2, 7, 8, 14, 20, 44]), we propose an evaluation metric with 13 desirable properties in terms of security and usability, enabling PAKE protocols to be systematically rated across a common spectrum.

2.3.1 Security benefits.

- S1. Key agreement.** The user and the server can negotiate a common session key at the end of the protocol execution. The consistency of a session key implies the authenticity of both parties, and it will protect subsequent data communication.
- S2. Mutual authentication.** The user and the server can verify the authenticity of each other. Considering the possible malicious/compromised authentication server, only user-to-server authentication is not enough, which may cause irreparable losses for users.
- S3. Explicit authentication.** It is often mentioned together with implicit authentication. Implicit authentication means that both parties may not notice the failure of the protocol until they use their session keys to interact with errors. Explicit authentication means that an honest party can immediately know whether the protocol succeeded or not, at the end of PAKE.
- S4. Perfect forward secrecy.** It can be divided into weak forward secrecy (wFS) and perfect forward secrecy (PFS) according to whether the attacker has actively participated in the protocol. PFS means that even if the long-term private key of one or more parties is stolen by an attacker, the security of previous session keys is still not threatened [14].
- S5. User anonymity.** The protocol protects user identity and achieves the untraceability of user activities [46]. In other words, an attacker cannot confirm the user's identity from the protocol messages or distinguish whether two independent sessions are from the same user.
- S6. Resistance to known attacks.** The PAKE protocol needs to resist various known attacks. It is a basic requirement for security. We summarize eleven possible attacks.
 - (1) *Replay attack.* This attack captures the freshness of messages. An attacker can replay (possibly modify) previous messages to spoof legitimate users. If the previous message replayed by the attacker has not been detected by legitimate users until the end of the PAKE protocol execution, the replay attack is considered successful.
 - (2) *Man-in-the-middle attack.* An active "man-in-the-middle" attacker can manipulate the session key without being detected by intercepting, modifying, delaying, and interleaving messages between honest communication entities.
 - (3) *Offline password guessing attack.* An attacker can verify the correctness of the guessing password offline by utilizing protocol messages between legitimate users. The assumption of the attacker's ability here does not involve the compromise of long-term private keys and password files, but only focuses on the eavesdropping of protocol messages.
 - (4) *Undetectable online password guessing attacks.* An attacker can eliminate multiple bad passwords in a single online attempt. In addition, there are two special attacks against RSA-based PAKE protocols that combine offline guessing with online guessing, i.e., *e-residue attacks* [23] and *separation attacks* [59].
 - (5) *Unknown key-share attack.* An attacker forces honest parties into establishing a session key where at least one party does not know that the session key is shared with others.
 - (6) *Parallel-session attack.* This attack works when the user is engaged in several sessions in parallel with another user. In two parallel sessions with the same user, the attacker acts as the recipient of the first session and the initiator of the second session. The attacker can pass authentication by relaying the sender's message back to the sender.
 - (7) *Known-key attack.* An attacker who knows a session key established between legitimate users can compute other secrets (e.g., the password). A known-key attack emphasizes the compromise of the session key, not the long-term private key or password file.
 - (8) *Ephemeral secret leakage attack.* An attacker reveals ephemeral session key materials of both parties and tries to learn the session key [60]. For example, if the random-number generator is corrupted, the attacker can learn random numbers selected by the party.

- (9) *Key compromise impersonation attack*. This attack captures the compromise of the long-term private key of parties. After the long-term key of the victim is compromised, an attacker will inevitably be able to impersonate the victim, which is also known as the impersonation attack. But key compromise impersonation (KCI) attacks highlight that the attacker can cheat the victim by impersonating other entities.
- (10) *Pre-computation attack*. An attacker can pre-compute all passwords in the dictionary (such as a reverse lookup table from the hash to the plaintext password), so that they can find the password immediately if the server is compromised.
- (11) *Quantum attack*. This attack captures the ability of quantum adversaries to efficiently solve the classical hardness assumptions (e.g., integer factorization and discrete logarithm) underlying PAKE. Although significant progress has been made in developing quantum algorithms with exponential speedups [61], the most impactful quantum attacks to date are those leveraging Shor's algorithm [62] and Grover's algorithm [63]. A quantum adversary equipped with a large-scale quantum computer can run Shor's algorithm to break PAKEs (e.g., OPAQUE [7] and KHAPE [11]) based on classical hardness assumptions, and employ Grover's algorithm to accelerate brute-force search and hash collision attacks.

In practice, denial-of-service (DoS) attacks represent a common threat in which adversaries flood servers with authentication requests in an attempt to exhaust system resources. To mitigate such attacks, Bresson et al. [64] proposed the OMDHKE, which incorporates a client-side proof-of-work mechanism. However, this defense is a general mechanism for a broad range of cryptographic protocols rather than an inherent feature of PAKE design. Moreover, proof-of-work imposes significant computational overhead on clients, thereby reducing usability. Fundamentally, resistance to DoS attacks is a broader concern for all interactive systems and is not unique to PAKE protocols. Service providers are expected to implement non-cryptographic protections (such as rate limiting, CAPTCHA challenges, or account lockout mechanisms) that lie outside the scope of cryptographic protocol design. For this reason, we exclude DoS resistance from our evaluation criteria for PAKEs.

2.3.2 Usability benefits.

- U1. Password friendly.** A user can freely choose their password and have no other burden besides memorizing it, including the ability to update the password at any time after initial registration. The high-entropy key generated by the system offers higher security, but it compromises the vital motivation of users to choose password authentication, i.e., memorability and deployability. Hence, weak passwords are a fact of life that we must face. The goal of password authentication researchers is not to prevent users from using weak passwords, but to find suitable methods for achieving authentication using weak passwords.
- U2. Login device friendly.** The practical deployment of PAKE requires a client, such as a browser plugin, a mobile software [14, 65]. Users should be able to log in on any client configured with a PAKE. It means that the client should only store system public parameters (e.g., group elements in J-PAKE [16] and common reference strings in SPAKE2 [36]).
- U3. Support pre-computation.** The client or the server can pre-compute parameters that don't need to wait until the other party's message is received. For example, the PAKE using the Diffie-Hellman key exchange (e.g., [7, 14, 15]) can pre-compute g^x, g^y , where g is the public base for exponentiation and $\{x, y\}$ are random numbers of client and the server, respectively. Using this trick, the computational effort of the participants and the execution time of the protocol could be significantly reduced (see Table 10).
- U4. Storage overhead.** The storage overhead is quantified as the bit-lengths of the necessary public parameters, key materials, identifiers, password files, etc. Note that when two users

share a password directly (e.g., J-PAKE [16]), neither party needs to store it. Considering the memory capability of the participants, the storage overhead should be as low as possible.

- U5. Computation overhead.** The computation overhead is quantified as the time cost for the participants to execute the protocol algorithm, which can affect the total running time. In particular, these values that can be pre-computed have no effect. As a result, if a PAKE protocol supports pre-computation, we only need to consider the execution of the protocol algorithm after pre-computation. Typically, low computation overhead is expected.
- U6. Transmission overhead.** The transmission overhead is quantified as the bit length of the transmitted data for the participants in executing the protocol. Note that some PAKE protocols (e.g., [8]) omit the participant's identifier in message transmission, but it is still part of the protocol. Heavy communication overhead should be avoided.
- U7. Round complexity.** A protocol flow refers to the data stream sent by a participant each time. If two participants can each send a protocol flow in no order, the two flows are considered as one round (e.g., [15, 16]). Otherwise, they are considered as two rounds (e.g., [7, 10]). A large number of protocol rounds will increase the running time. Round complexity is affected by explicit/implicit authentication. For example, if one-round PAKE achieves mutually implicit authentication (e.g., PPK [34] and SPAKE2 [36]), it is round-optimal.

3 TAXONOMY OF PAKE PROTOCOLS

We now provide a taxonomy of PAKE protocols in terms of the design strategy (see Fig. 2). Our taxonomy highlights the general trend and emphasizes their commonalities in design strategy. To keep our survey general and simple, we only select representative PAKE protocols to be included in Tables 2~7. In addition, we only discuss how such protocols achieve authentication and key exchange here, and leave the discussion of all aspects of each protocol individually later (see Tables 9 and 10). For simplicity, we employ “password” in Types I~VI instead of “the password itself or the mapping of the password”. For example, in Type-I Password as an encryption key, the KHAPE protocol [11] uses the password as an encryption key, and the OKAPE-HMQV protocol [8] uses the hash of the password as the encryption key (see Table 2).

Type-I. Password as an encryption key. The password is used to derive an encryption key. The message to be encrypted is usually an ephemeral public key. Only the party holding the password can decrypt the public key. Examples: KHAPE [11], OKAPE-HMQV [8].

Type-II. Password as a generator. The password is used to derive a generator. Each party raises it to the power of an ephemeral private key. Participants prove their knowledge of the password by deriving the common generator. Examples: SPEKE [15], P-SPEKE [20].

Type-III. Password as an exponent of common reference strings. In this type, there are one or more common reference strings. Each party raises it to the power of the password, and then multiplies it by an ephemeral public key. Examples: SPAKE2 [36], PFS-SPAKE2 [14].

Type-IV. Password as an element of the multiplication operation. The protocol uses a hash-to-group (or hash-to-curve) function to map the password to a group element, then multiply by an ephemeral public key. Examples: PAK [34], PAK-R [66], and PAK-Z [67].

Type-V. Password as a mask for messages. The password is used as an exponent to mask the ephemeral public key. There is no need for a common reference string or a hash-to-group function, and only public key technology is used. Examples: SRP [10], AMP [38].

Type-VI. Password as a parameter of smooth projective hashing. In this type, the session key is computed based on the properties of the smooth projective hash function, where the password is used as a parameter. Examples: Katz-Vaikuntanathan [68], Groce-Katz [56].

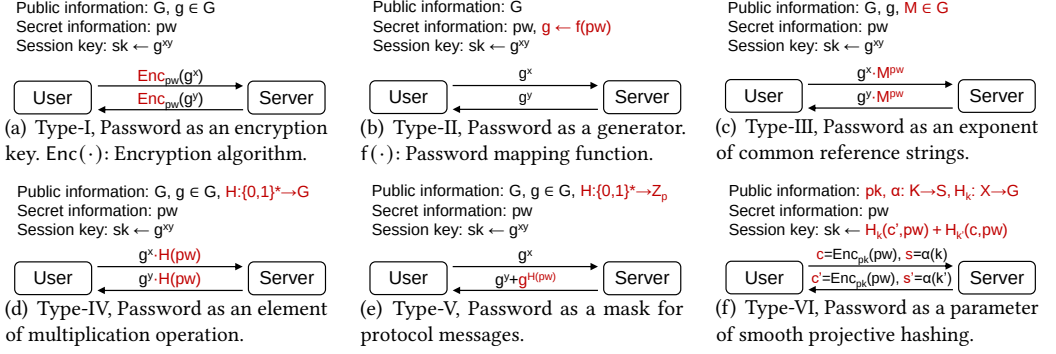


Fig. 2. Overview of design strategies for Types I–VI. We provide four parts: public information, secret information, session keys, and protocol messages. The characteristics of each type are marked in red. Note that for simplicity and intuition, both parties share a password, and the details of the session key are omitted.

Type-VII. Others. The design strategy differs from the above six types, and there is also no uniform or regular usage of the password. Roughly speaking, the password is used as a bit string (but with different roles) in [50, 69, 70] and to replace the private key of a trusted third party [71].

3.1 Type-I: Password as an Encryption Key

Bellovin and Merritt [13] presented three variants of EKE: Diffie-Hellman EKE (DH-EKE), RSA-EKE, and ElGamal-EKE. Researchers have made various extensions and improvements to DH-EKE (e.g., [8, 11, 72]) and RSA-EKE (e.g., [35, 73, 74]), while ElGamal-EKE is less common. A plausible reason is that the defined group G usually can be a finite group, or an elliptic curve group [75]. Since the exponentiation operation and the point multiplication operation in elliptic curve cryptosystems play similar roles, a PAKE protocol in the DH setting can generally also be implemented in the ECC setting. Table 2 summarizes PAKE protocols that use the same paradigm as EKE [13].

3.1.1 DH-EKE. In Bellovin-Merritt’s work [13], the EKE protocol requires both parties to hold a plaintext password. This causes passwords to be exposed immediately after the server is compromised. To address this problem, Bellovin and Merritt [37] extended the EKE protocol to the Augmented Encrypted Key Exchange (A-EKE) protocol, the first verifier-based/asymmetric PAKE protocol. In order to reduce the relative complexity of EKE [13], Steiner et al. [76] proposed the minimal EKE (M-EKE) protocol. M-EKE only requires three messages and three encryption operations over a single data block. Similarly, considering the complexity, Bellare and Rogaway [72] proposed AuthA with lower cost in communications and greater versatility than EKE [13]. Later, Bresson et al. [77] modeled AuthA as the One-Encryption Key-Exchange (OEKE) protocol. Then, the German Federal Office for Information Security (BSI) proposed the Password-Authenticated Connection Establishment (PACE v1) protocol and its updated version, PACE v2 [78]. The participants of PACE v1/v2 are a chip contained in a machine-readable travel document and a reader. The chip is required to send a random nonce encrypted with the password to the terminal.

Gentry et al. [79] proposed the Ω -method to convert a symmetric PAKE to an asymmetric PAKE. The Ω -method requires two additional communication rounds and one signature computation/verification. More specifically, the server sends a private key sk_{sign} that is double encrypted with a password pw and a temporary session key sk , i.e., $E_{sk}(E'_{pw}(sk_{sign}))$. Then the client decrypts the sk_{sign} and sends the protocol transcript signed with sk_{sign} . Then, Hwang et al. [9] proposed a method for converting a symmetric PAKE to an asymmetric PAKE, which requires only one additional round. More specifically, the client hashes $Y^{H(pw)}$ together with a temporary session

Table 2. PAKE protocols using the password as an encryption key (Type-I).[†]

Protocol	Year	Reference	Sym./Asym.	Plaintext	Key for encryption/decryption	Ciphertext
BareEKE	2024	CRYPTO [80]	Symmetric	$m = g^x$	$k = pw$	$c = (E_k(m), id)$
SweetPAKE	2024	AisaCCS [81]	Symmetric	$m = g^x$	$k = pw$	$c = E_k(m)$
FuzzyPAKE	2023	ASIACRYPT [82]	Symmetric	$m = YGC(x)$	$k = pw$	$c = E_k(OT(m))$
OKAPE-HMQV	2022	EUROCRYPT [8]	Asymmetric	$m = g^y$	$k = H(pw)$	$c = E_k(m)$
aEKE-HMQV	2022	EUROCRYPT [8]	Asymmetric	$m = g^y$	$k = H(pw)$	$c = E_k(m)$
KHAPE	2021	CRYPTO [11]	Asymmetric	$m = (a, g^b)$	$k = pw$	$c = E_k(m)$
Hwang et al.	2018	SCN [9]	Asymmetric	$m = g^y$	$k = sid H(pw)$	$c = E_k(m)$
PACE v2	2008	BSI report [51]	Symmetric	$m = s \in \{0, 1\}^{\ell_1}$	$k = pw$	$c = E_k(m)$
PACE v1	2006	BSI report [83]	Symmetric	$m = s \in \mathbb{Z}_q$	$k = H(pw 0)$	$c = E_k(m)$
Ω -method	2006	CRYPTO [79]	Asymmetric	$m = sk_{sign}$	$k_1 = sk, k_2 = pw$	$c = E_{k_1}(E'_{k_2}(m))$
OEKE	2003	ACM CCS [77]	Symmetric	$m = g^y$	$k = H(pw)$	$c = E_k(m)$
AuthA	2000	IEEE P1363 [72]	Asymmetric	$m_1 = g^x, m_2 = g^y$	$k = H(pw)$	$c_1 = E_k(m_1), c_2 = E_k(m_2)$
M-EKE	1995	ACM SIGOPS [76]	Symmetric	$m = g^x$	$k = pw$	$c = E_k(m)$
A-EKE	1993	ACM CCS [37]	Asymmetric	$m_1 = g^x, m_2 = g^y$	$k = H(pw)$	$c_1 = E_k(m_1), c_2 = E_k(m_2)$
DH-EKE	1992	IEEE S&P [13]	Symmetric	$m_1 = g^x, m_2 = g^y$	$k = pw$	$c_1 = E_k(m_1), c_2 = E_k(m_2)$
CPEKEP	2004	ASIACRYPT [35]	Symmetric	$m = a \in_R \mathbb{Z}_n^*$	$k = H(n e r_U r_S pw)$	$c = E^{r^{-1}}(k \cdot E(m)), r \in (0, 80]$
PEKEP	2004	ASIACRYPT [35]	Symmetric	$m = a \in_R \mathbb{Z}_n^*$	$k = H(n e r_U r_S pw)$	$c = E^r(k \cdot E(m)), r = \lfloor \log_2 n \rfloor$
SNAPI-X	2000	ASIACRYPT [74]	Symmetric	$m = a \in_R \mathbb{Z}_n^*$	$k = H(n e r_U r_S g^{H(ID_U ID_S pw)})$	$c = k \cdot E(m)$
SNAPI	2000	ASIACRYPT [74]	Symmetric	$m = a \in_R \mathbb{Z}_n^*$	$k = H(n e r_U r_S pw)$	$c = k \cdot E(m)$
OKE	1997	SPW [73]	Symmetric	$m = a \in_R \mathbb{Z}_n^*$	$k = H(n e r_U r_S pw)$	$c = k \cdot E(m)$
RSA-EKE	1992	IEEE S&P [13]	Symmetric	$m = e$	$k = pw$	$c = k \cdot E(m)$

[†] This table shows the plaintext (i.e., m, m_1, m_2), key (i.e., k, k_1, k_2), and ciphertext (i.e., c, c_1, c_2). $E(\cdot)$, $E'(\cdot)$ are encryption algorithms. n, e is the RSA public key. $YGC(\cdot)$ denotes Yao's garbled circuits, and $OT(\cdot)$ denotes oblivious transfer. sid is the session identifier, and sk is the session key. $\{x, r_U\}$ and $\{y, r_S\}$ are the ephemeral secret keys selected by the user U and the server S , respectively. a, b are private keys used in HMQV [39]. pw is the user's password. $H(\cdot)$ is a hash function. $||$ denotes string concatenation. The DH-based PAKEs appear above the dash, the RSA PAKEs below.

key sk to derive an authenticator $t = H(Y^{H(pw)} || Y || sk || \text{transcript})$, where transcript is a transcript of protocol messages. After receiving the authenticator t from the client, the server will first construct t' by using the verifier $V = g^{H(pw)}$ in the password file, and then check whether $t = t'$. At CRYPTO'21, Gu et al. [11] proposed the Key-Hiding Asymmetric PAKE (KHAPE) protocol. KHAPE [11] and EKE [13] are different in encryption: EKE [13] uses the password to encrypt each *unauthenticated* key exchange (KE) message and requires the KE messages must be random in some message space; KHAPE [11] uses the password to encrypt an *authenticated* key exchange (AKE) message and requires the underlying AKE to have the key-hiding property. Motivated by low round complexity, Santos et al. [8] proposed aEKE-HMQV and OKAPE-HMQV, which use the password to encrypt a *unilaterally authenticated* key exchange message. The aEKE-HMQV uses an unsalted password hash and takes a single simultaneous flow, while OKAPE-HMQV uses a salted password hash and takes two rounds.

3.1.2 RSA-EKE. The RSA-EKE protocol proposed by Bellare and Merritt [13] has two apparent constraints: (1) A new RSA key pair (i.e., n, e and d) needs to be generated for each protocol execution; (2) Users need first to encrypt the RSA public key e and then send it. To overcome these issues, Lucks [73] introduced the OKE protocol, so that the generated RSA key pair can once be reused and the public key e is sent openly. Then, Mackenzie et al. [74] pointed out that an e -residue attack is feasible for the OKE protocol, because the receiver can't check whether the RSA public exponent e satisfies the equation $\gcd(e, \phi(n)) = 1$ without knowing the factorization of RSA modulus n . To thwart this attack, Mackenzie et al. [74] introduced SNAPI and SNAPI-X, which require the exponent e ($e > n$) to be a prime number to make the equation $\gcd(e, \phi(n)) = 1$ true. Later, Zhang [35] presented PEKEP and CPEKEP, which use multiple RSA encryption (i.e., r times in Table 2) to resist e -residue attacks, thereby eliminating the assumption that the RSA public exponent must be a large prime number. Then, Wei et al. [84] extended PEKEP to a gateway-oriented PAKE protocol, named RSA-GPAKE. RSA-GPAKE establishes an authenticated session key between the user and the gateway. Subsequently, Wang et al. [85] pointed out that the RSA-GPAKE protocol is

Table 3. PAKE protocols using the password as a generator (Type-II).[†]

Protocol	Year	Reference	Sym./Asym.	Generator	User→Server*	Server→User
BareCPace	2024	CRYPTO [80]	Symmetric	$g = \text{Map2Point}(H(sid \ g_1^{\text{PBKDF}(ID_U \ pw)}))$	$X = g^x$	$Y = g^y$
AucPace	2019	IACR Trans. [90]	Asymmetric	$g = \text{Map2Point}(H(sid \ g_1^{x \cdot \text{PBKDF}(ID_U \ pw \ salt)}))$	$X = g^x$	$Y = g^{xy}$
CPace	2019	IACR Trans. [90]	Symmetric	$g = \text{Map2Point}(H(sid \ g_1^{\text{PBKDF}(ID_U \ pw)}))$	$X = g^x$	$Y = g^y$
P-SPEKE	2018	IEEE TIFS [20]	Symmetric	$g = H(pw)^2$	$X = g^x$	$Y = g^y$
OPAQUE	2018	EUROCRYPT [7]	Asymmetric	$g = H(pw)$	$X = g^x$	$Y^* = g^{xy}$
DE-PAKE	2016	AsiaCCS [65]	Asymmetric	$g = H(pw)$	$X = g^x$	$Y^* = g^{xy}$
Patched SPEKE	2014	SSR [88]	Symmetric	$g = H(pw)^2$	$X = g^x$	$Y = g^y$
B-SPEKE	1997	WETICE [87]	Asymmetric	$g = H(pw)^2$	$X = g^x$	$Y = g^y$
SPEKE	1996	CCR [15]	Symmetric	$g = pw^2$	$X = g^x$	$Y = g^y$

[†] $H(\cdot)$ is a hash-to-group function. The base g is computed by each party and is not public, while g_1 is a public generator. Due to space constraints, we omit protocol messages that are not related to g . For example, we only show a part of the OPAQUE protocol [7], namely, the Oblivious Pseudo-Random Function (OPRF) interaction. This is because the use of OPRF to resist pre-computation attacks is the most significant feature of the OPAQUE protocol.

* User→Server denotes the message sent from the user to the server, while Server→User denotes the message sent from the server to the user. We use $X = g^x$ and $Y = g^y$ to denote the ephemeral Diffie-Hellman public keys, and X^* , Y^* to denote the values after further operations (such as exponents). Table 4~7 also uses the above symbols.

vulnerable to separation attacks [59], and put forward the RSA-GPAKE+ protocol. RSA-GPAKE+ is inspired by the idea of avoiding information leakage in Zhang's work [35].

3.2 Type-II: Password as a Generator

In 1996, Jablon [15] proposed a simple password exponential key exchange (SPEKE) protocol, and discussed three attacks related to the SPEKE protocol: discrete log computation, leaking information, and small subgroup confinement. Informally, both SPEKE [15] and DH-EKE [13] can be regarded as slight modifications to the basic Diffie-Hellman key exchange [86]. However, in the protocol design, even the slightest modification can have profound effects [16]. DH-EKE [13] encrypts key exchange messages by using a password as a symmetric key. Yet, SPEKE [15] converts the password into a base for exponentiation, which is the common feature of Type-2.

Then, Jablon [87] proposed an extension of SPEKE (i.e., B-SPEKE) to prevent user passwords from being exposed directly after the server is compromised. In addition to hashing the password, B-SPEKE [87] adds a second Diffie-Hellman exchange to prove the participant's knowledge of the password. Later, Hao and Shahandashti [88] presented two attacks on the SPEKE protocol [87]: an impersonation attack and a key malleability attack. To resist these two attacks, Hao and Shahandashti [88] suggested adding communication messages to the computation of session keys, where the communication messages include not only the identifiers of both parties, but also the ephemeral Diffie-Hellman exchange values. Then, Hao et al. [20] proposed a P-SPEKE protocol based on the idea from [88], which has been included in the ISO/IEC 11770-4 [89].

In 2019, Haase and Labrique [90] proposed cPACE and AucPace tailored for the Internet of Things. The generation of the base in [90] includes the session identifier, the user's identity, and the password. Besides, both DE-PAKE [65] and OPAQUE [7] use an Oblivious Pseudo-Random Function (OPRF) to provide security upon server compromise. To retrieve the OPRF value $H'(pw, H(pw)^y)$, the client raises the hashed value $H(pw)$ to a random exponent x , and sends $X = H(pw)^x$ to the server (see Table 3). The server checks that $X \in \mathbb{G}$ and raises it to the secret exponent y , i.e., $Y^* = g^{xy}$. Finally, the client completes the OPRF computation by computing $H'(pw, Y^{1/x})$.

3.3 Type-III: Password as an Exponent of Common Reference Strings

To eliminate the random oracle assumption, Katz et al. [91] proposed a PAKE protocol, commonly referred to as KOY, at EUROCRYPT 2001, which operates solely in the standard model. The KOY protocol assumes one set of common parameters available to all parties. Then, Kobara and Imai [92]

Table 4. PAKE protocols using the password as an exponent of common reference string (Type-III).[†]

Protocol	Year	Reference	Sym./Asym.	Public para.	User→Server	Server→User
KC-SPAKE2+	2020	TCC [95]	Asymmetric	$g, M \in \mathbb{G}$	$X^* = g^x \cdot M^{H(ID_U \ ID_S \ pw)}$	$Y = g^y$
KC-SPAKE2	2020	TCC [95]	Symmetric	$g, M \in \mathbb{G}$	$X^* = g^x \cdot M^{pw}$	$Y = g^y$
saPAKE-BB	2019	CRYPTO [58]	Asymmetric	$g_1, g_2, M \in \mathbb{G}$	$X_1^* = g_1^x \cdot M^{H(pw)}, X_2^* = g_2^x$	$Y^* = (g_2^{H(pw)/(salt+H(pw))})^{y_1} \cdot (X_1^*)^{y_2} \cdot (X_2^*)^{y_3}$
PFS-SPAKE2	2018	ProvSec [14]	Symmetric	$g, M \in \mathbb{G}$	$X^* = g^x \cdot M^{pw}$	$Y = g^y$
VTBPEKE	2017	AsiaCCS [49]	Asymmetric	$g, M \in \mathbb{G}$	$X^* = g^x \cdot M^{x \cdot H(salt \ pw)}$	$Y^* = g^y \cdot M^{y \cdot H(salt \ pw)}$
TBPEKE	2017	AsiaCCS [49]	Symmetric	$g, M \in \mathbb{G}$	$X^* = g^x \cdot M^{x \cdot pw}$	$Y^* = g^y \cdot M^{y \cdot pw}$
SESPAKE	2017	RFC 8133 [96]	Asymmetric	$P, Q \in E$	$X^* = x^*P - H(ID_U \ pw)^*Q$	$Y^* = y^*P + H(ID_U \ pw)^*Q$
SPAKE2+	2008	EUROCRYPT [94]	Asymmetric	$g, M, N \in \mathbb{G}$	$X^* = g^x \cdot M^{H(ID_U \ ID_S \ pw)}$	$Y^* = g^y \cdot N^{H(ID_U \ ID_S \ pw)}$
SPAKE2	2005	CT-RSA [36]	Symmetric	$g, M, N \in \mathbb{G}$	$X^* = g^x \cdot M^{pw}$	$Y^* = g^y \cdot N^{pw}$
SPAKE1	2005	CT-RSA [36]	Symmetric	$g, M, N \in \mathbb{G}$	$X^* = g^x \cdot M^{pw}$	$Y^* = g^y \cdot N^{pw}$
Jiang-Gong	2004	SAC [93]	Symmetric	$g, M \in \mathbb{G}$	$X^* = g^x \cdot M^{pw}, M^x$	$Y^* = g^{y_1} \cdot M^{y_2}$
Kobara-Imai	2002	IEICE Trans. [92]	Symmetric	$g, M \in \mathbb{G}$	$X^* = g^x \cdot M^{pw}$	$Y^* = g^y \cdot M^{pw}$
KOY	2001	EUROCRYPT [91]	Symmetric	$g, M \in \mathbb{G}$	$X^* = g^x \cdot M^{pw}$	$Y^* = g^y \cdot M^{pw}$

[†] For simplicity and better intuition, we omit other values in the protocol message that are not related to the common reference string, e.g., $\{y, y_1, y_2, y_3\}$ and $\{x, x_1, x_2, x_3\}$ are the ephemeral secret keys selected by the user U and the server S , respectively. In addition, P and Q are two points on the elliptic curve E .

reduced the computational cost of KOY: Only about 2.34 modular exponentiations are required in [92], while more than 6.5 modular exponentiations are required in KOY [91]. Jiang and Gong [93] optimized the communication round of KOY: Only three rounds are required to achieve explicit mutual authentication in [93], while four rounds are required in KOY [91].

Abdalla and Pointcheval [36] believed that, although KOY [91] completely eliminates the need of random oracles, it is less efficient than EKE [13] and is vulnerable to man-in-the-middle attacks. Therefore, for efficiency reasons, Abdalla and Pointcheval [36] presented two protocols: SPAKE1 and SPAKE2 (see Table 4). Both protocols are comparable in efficiency to any existing EKE protocol and only require one random oracle instance. In order to provide security upon server compromise, Cash et al. [94] proposed an improved version (i.e., SPAKE2+) which uses a password-derived value $H(ID_U \| ID_S \| pw)$, instead of pw as the exponent (see Table 4). Additionally, in order to satisfy perfect forward secrecy, Becerra et al. [14] proposed a PFS-SPAKE2 protocol by incorporating key-confirmation codes in SPAKE2 [36]. Then, Shoup [95] proposed two protocols: KC-SPAKE2 and KC-SPAKE2+. KC-SPAKE2 is essentially the PFS-SPAKE2 protocol [14], and Shoup's [95] contribution is to prove the security of KC-SPAKE2 in the Universal Composable (UC) framework. KC-SPAKE2+ incorporates standard key confirmation flows in SPAKE2+ [94].

Inspired by SPAKE2 [36], Pointcheval and Wang [49] proposed the TBPEKE and VTBPEKE protocols. Both protocols achieve user authentication to the server with three messages. In 2017, Smyshlyaev [96] specified the Security Assessment Standardized PAKE protocol (i.e., SESPAKE). Both parties use password-based coefficients multiplied by one of the predefined elliptic curve points (i.e., $H(ID_U \| pw)^*Q$) to mask the Diffie-Hellman key exchange. At CRYPTO'19, Bradley et al. [58] proposed the saPAKE-BB protocol that satisfies the notion of strong asymmetric PAKE [7].

3.4 Type-IV: Password as an Element of Multiplication Operation

At EUROCRYPT'00, Boyko et al. [34] proposed a PAK protocol. Compared to AuthA [43], PAK only relies on random oracles, while AuthA relies on random oracles and ideal block ciphers. PPK and PAK-X are two variants of PAK: PPK can achieve implicit authentication within two rounds; PAK-X requires the server to store a verifier for the password and is resilient to server compromise. Then, MacKenzie [66] proposed a revised PAK protocol called PAK-R, with a client-side computation of only half that of PAK [34]. PAK-RY is an asymmetric PAKE protocol used to replace PAK-X [34]: the first two messages are the same as PAK-X, while in the third message, the client of PAK-RY simply proves knowledge of the password. Later, MacKenzie [67] proposed a general method for making PAK protocols (e.g., PAK, PPK) resilient to server compromise. Applying this method to

Table 5. PAKE protocols using the password as an element of multiplication operation (Type-IV).[†]

Protocol	Year	Reference	Sym./Asym.	Hash function	User→Server	Server→User
McQuoid et al.	2020	ACM CCS [100]	Symmetric	$H : \{0, 1\}^* \rightarrow E$	$X^* = g^x \cdot H(pw \ r_U)^{-1}$	$Y^* = g^y \cdot H(pw \ r_S)^{-1}$
SAE	2008	SensorComm [98]	Asymmetric	$H : \{0, 1\}^* \rightarrow E$	$X^* = x \cdot H(ID_U \ ID_S \ pw)$	$Y^* = y \cdot H(ID_U \ ID_S \ pw)$
PAK-Z+	2005	IEEE P1363 [97]	Asymmetric	$H : \{0, 1\}^* \rightarrow G$	$X^* = g^x \cdot H(ID_U \ ID_S \ pw)$	$Y = g^y$
PAK-Z	2002	IEEE P1363 [67]	Asymmetric	$H : \{0, 1\}^* \rightarrow G$	$X^* = g^x \cdot H(ID_U \ pw)$	$Y = g^y$
PAK-RY	2001	CT-RSA [66]	Asymmetric	$H : \{0, 1\}^* \rightarrow \{0, 1\}^q$	$X^* = g^{x_1} \cdot x_2^y \cdot H(ID_U \ ID_S \ g^{H(\min ID \ \max ID \ pw)})$	$Y = g^y$
PAK-R	2001	CT-RSA [66]	Symmetric	$H : \{0, 1\}^* \rightarrow \{0, 1\}^q$	$X^* = g^{x_1} \cdot x_2^y \cdot H(ID_U \ ID_S \ pw)$	$Y = g^y$
PAK-X	2000	EUROCRYPT [34]	Asymmetric	$H : \{0, 1\}^* \rightarrow \{0, 1\}^q$	$X^* = g^x \cdot H(ID_U \ ID_S \ pw)^r$	$Y = g^y$
PPK	2000	EUROCRYPT [34]	Symmetric	$H : \{0, 1\}^* \rightarrow \{0, 1\}^q$	$X^* = g^x \cdot H(ID_U \ ID_S \ pw)^r$	$Y^* = g^y \cdot (H(ID_U \ ID_S \ pw)^r)$
PAK	2000	EUROCRYPT [34]	Symmetric	$H : \{0, 1\}^* \rightarrow \{0, 1\}^q$	$X^* = g^x \cdot H(ID_U \ ID_S \ pw)^r$, where $p = rq + 1$	$Y = g^y$

[†] The protocol message is usually the product of an ephemeral Diffie-Hellman public key with a hash of the password. g is a generator of a subgroup of $\mathbb{Z}_q^*$ of size q , which is public to all parties. $p = rq + 1$ for some value r co-prime to q . $\{x, x_1, x_2, r_U\}$ and $\{y, r_S\}$ are the ephemeral secret keys selected by the user U and the server S , respectively. $\min ID / \max ID$ is the min/max value between the user's identity ID_U and the server's identity ID_S .

PAK [34] resulted in the PAK-Z protocol. But Gentry et al. [97] pointed out that in fact, PAK-Z is not resilient to server compromise and then proposed an improved version, i.e., PAK-Z+.

At PKC'04, Bresson et al. [64] for the first time proved that the product of the message with a hash of the password (e.g., $X^* = g^x \cdot H(pw)$) is secure. In addition, Bresson et al. [64] also proposed the OMDHKE protocol that can resist denial of service attacks. Then, Harkins [98] proposed the Simultaneous Authentication of Equals (SAE) protocol. SAE has been implemented for IEEE 802.11 based mesh networks. SAE requires a finite cyclic group: prime modulus groups or elliptic curve groups [99]. At CCS'20, McQuoid et al. [100] proposed a symmetric PAKE with optimal round complexity and computational efficiency: one round to achieve implicit authentication, and only two exponentiations per party. They employ the primitive of programmable-once public function.

3.5 Type-V: Password as a Mask for Messages

At NDSS'98, Wu [10] introduced the Secure Remote Password (SRP) protocol, where the server stores a mapping of the user password, namely $V = g^{H(ID_U \| pw \| salt)}$. It should be noted that the SRP protocol uses a mathematical expression $Y^* = g^y + V$, a novel construction that has never appeared in previous schemes. A user needs to compute $g^y \leftarrow Y^* - g^{H(ID_U \| pw \| salt)}$ to prove the knowledge of the password. Wu [21] pointed out two issues with the SRP protocol [10]. The first is an attack on the security. An attacker impersonates the server to send the exchanged key value, and replaces the original verifier and the ephemeral key value with two guessed passwords, thereby testing two possible passwords in an impersonation attack. The second one is an operational problem. That is, the server needs to wait for the user to send the exchanged key value before sending the random parameter u . In order to solve these two problems, Wu [21] proposed the SRP-6 protocol as an improved version of SRP [10], which uses the mathematical expression $Y^* = g^y + k \cdot V$ (e.g., $k = 3$) to eliminate symmetry, and redefines $u = H(X^* \| Y^*)$ to remove the message order limitation.

At NDSS'01, Kwon [38] proposed the AMP protocol, where the server stores $V = g^{H(ID_U \| pw)}$. The server sends $Y^* = (g^x \cdot V)^y$ to the user and requires the user to compute $g^y \leftarrow (Y^*)^{1/(x+H(ID_U \| pw))}$. The AugPAKE protocol [101] adopts a design strategy similar to AMP [38]. That is, the server requires the user to compute $g^{H(y)} \leftarrow (Y^*)^{1/(x+pw \cdot H(ID_U \| ID_S \| X))}$. For more details, see Table 6.

At SPW'08, Hao and Ryan [16] proposed the password-authenticated key exchange by juggling (short for J-PAKE) protocol. J-PAKE has been included in ISO/IEC 11770-4 (2017) [89] as an international standard and formally published in RFC 8236 [102]. In particular, J-PAKE was implemented in OpenSSL and OpenSSH as an experimental authentication protocol [103]. In addition, it was used by Firefox Sync version 1.1 [19] and continues to be part of the Pale Moon Sync service [104]. In Hao-Ryan's work [16], participants first exchange their generated ephemeral public keys, such as standard Diffie-Hellman key exchange, and then combine these ephemeral public keys with a shared password in an additional round. The technique adopted by the J-PAKE

Table 6. PAKE protocols using the password to mask protocol messages (Type-V).[†]

Protocol	Year	Reference	Sym./Asym.	User→Server	Server→User	Session key material
AugPAKE	2010	RFC 6628 [101]	Asymmetric	$X = g^x$	$Y^* = g^{x+H(y)} \cdot g^{pw \cdot H(ID_U \ ID_S \ X) \cdot H(y)}$	$\sigma = g^{H(y)}$
J-PAKE	2008	SPW [16]	Symmetric	$X^* = (g^{y_1} g^{y_2} g^{x_1})^{x_2} \cdot pw$	$Y^* = (g^{x_1} g^{x_2} g^{y_1})^{y_2} \cdot pw$	$\sigma = g^{(x_1+y_1) \cdot x_2 \cdot y_2 \cdot pw}$
SRP-6	2002	IEEE P1363 [21]	Asymmetric	$X = g^x$	$Y^* = g^y + 3 \cdot g^{H(ID_U \ pw \ salt)}$	$\sigma = g^{y \cdot (x+H(X^* \ Y^*) \cdot H(ID_U \ pw \ salt))}$
AMP	2001	NDSS [38]	Asymmetric	$X = g^x$	$Y^* = g^{xy} \cdot g^{H(ID_U \ pw) \cdot y}$	$\sigma = g^{y \cdot (x+H(ID_U \ X^* \ Y^*))}$
SRP	1998	NDSS [10]	Asymmetric	$X = g^x$	$Y^* = g^y + g^{H(ID_U \ pw \ salt)}$	$\sigma = g^{y \cdot (x+u \cdot H(ID_U \ pw \ salt))}$
CHIP	2020	CRYPTO [71]	Asymmetric	$X = g^x, A = g^a$	$Y = g^y, B = g^b$	$\sigma = g^{(a+x+H(pw) \cdot H(ID_U \ A)) \cdot (b+y+H(pw) \cdot H(ID_S \ Y))}$

[†] In Fig. 2(e), we use SRP [10] as an example (see the symbol “+” in Fig. 2(e)) to show the protocol flows of Type-V. This table shows all methods to mask protocol messages. g is a generator of a subgroup of $\mathbb{Z}_q^*$ of size q , which is public to all parties. $\{x, x_1, x_2\}$ and $\{u, y, y_1, y_2\}$ are the ephemeral secret keys selected by U and S , respectively.

protocol is called “public key juggling”, that is, rearranging the public keys in a particular way, and encrypting the data by introducing quadratic terms on the exponents in a verifiable way.

3.6 Type-VI: Password as a Parameter of Smooth Projective Hashing

At EUROCRYPT’03, Gennaro and Lindell [40] found a novel application of smooth projective hash functions (SPHF) for password-based key exchange. Specifically, the client sends $\{c, s\}$ and the server sends $\{c', s'\}$, where c and c' are randomly generated commitments of pw , s and s' are two projection keys. Then, based on the (approximate) correctness and smoothness of SPHF, both parties compute the common session key material $H_k(c', pw) + H_{k'}(c, pw)$. Later, Canetti et al. [105] found that when simulating the Gennaro-Lindell protocol [40], it is impossible to ensure that the key computed by the adversary is the same as the key obtained from the functionality. To overcome this problem, Canetti et al. [105] added zero-knowledge proof to Gennaro-Lindell’s work [40]. Katz and Vaikuntanthan [68] introduced a CCA-secure (i.e., chosen-ciphertext attack secure) public-key encryption scheme based on the Learning With Errors (LWE) assumption, and then plugged this scheme into the Gennaro-Lindell framework [40] to present a generic three-round PAKE protocol. After that, Katz and Vaikuntanthan [30] introduced a one-round lattice-based PAKE protocol by combining the Non-Interactive Zero-Knowledge (NIZK) and the Approximate Smooth Projective Hashing (ASPH). Table 7 summarizes the representative PAKE protocols.

At CCS’10, Groce and Katz [56] proposed a new framework that relies on potentially weaker assumptions than the Gennaro-Lindell framework [40]: (1) a CCA-secure encryption scheme, and (2) a CPA-secure (i.e., chosen-plaintext attack security) encryption scheme with an associated smooth projective hash function. Then, Zhang and Yu [31] proposed a two-round lattice-based PAKE protocol by combining splittable public-key encryption and an adaptive ASPHF. Shin et al. [12] further proposed a two-round verifier-based PAKE using tamper-proof hardware. Recently, Li et al. [32] proposed an asymmetric PAKE protocol by employing smooth projective hash functions and commitment-based password-hashing schemes over lattice-based cryptography. At PKC’20, Jiang et al. [28] put forward two PAKE protocols based on the plain LWE and the Ring-LWE assumptions, which do not require any CCA-secure encryption or its variant. Later, Ren and Gu [29] proposed an asymmetric PAKE based on the Module-LWE assumption, which is a horizontal extension of the PAK protocol [67] in the field of module lattice.

3.7 Type-VII: Others

Three PAKE protocols [50, 69, 70] belong to *passwords as bit strings*. In addition, Cremers et al. [71] proposed two identity-based PAKE protocols that belong to *password as the private key of a trusted third party*.

- *Password as a bit string*. There are three PAKE protocols that use passwords as bit strings. At PKC’12, Canetti et al. [50] proposed a PAKE protocol based on Oblivious Transfer (OT). Specifically, the client acts as the OT receiver and uses the password pw as input. The server

Table 7. PAKE protocols using the password as a parameter of smooth projective hashing (Type-VI).[†]

Protocol	Year	Reference	Sym./Asym.	Session key material	Building blocks
Li et al.	2022	IEEE TDSC [32]	Asymmetric	$\sigma = H_k(c', H(\text{salt} pw)) + H_{k'}(c, r)$	SPHF + Micciancio-Peikert enc. [108]
H-VPAGE	2021	Comput. J. [12]	Asymmetric	$\sigma = H_k(c', pw) + H_{k'}(c, r)$	SPHF + Cramer-Shoup enc. [109]
Ren-Gu	2021	ICICS [29]	Asymmetric	$\sigma = \text{Rec}(s_c^T y_s, v)$	SPHF + Error reconciliation
LWE-PAKE	2020	PKC [28]	Symmetric	$\sigma = \mathcal{E}(E^T A s, v)$	SPHF + Key reconciliation
Zhang-Yu	2017	ASIACRYPT [31]	Symmetric	$\sigma = H_k(u', pw) + H_{k'}(u, pw)$	SPHF + Peikert enc. [110]
Katz-Vaikuntanathan	2011	TCC [30]	Symmetric	$\sigma = H_k(\text{label}', c', pw) + H_{k'}(\text{label}, c, pw)$	SPHF + Naor-Yung enc. [111]
Groce-Katz	2010	ACM CCS [56]	Symmetric	$\sigma = H_k(c', pw)$	SPHF + Kurosawa-Desmedt enc. [112]
Katz-Vaikuntanathan	2009	ASIACRYPT [68]	Symmetric	$\sigma = H_k(\text{label}', c', pw) + H_{k'}(\text{label}, c, pw)$	SPHF + Peikert enc. [110]
Canetti et al.	2005	EUROCRYPT [105]	Symmetric	$\sigma = H_k(c', pw) + H_{k'}(c, pw)$	SPHF + Shoup enc. [113]
Gennaro-Lindell	2003	EUROCRYPT [40]	Symmetric	$\sigma = H_k(c', pw) + H_{k'}(c, pw)$	SPHF + Cramer-Shoup enc. [109]

[†] We present the session key material σ in order to intuitively show the characteristics of session keys, which are different from Types I~V. Due to space constraints, the parameters will not be explained one by one. A Smooth Projective Hash Function (SPHF) is a collection of a keyed function $\{H_k : X \rightarrow \{0, 1\}^n\}_{k \in K}$, along with a projection function $\alpha : K \rightarrow S$, satisfying notions of correctness and smoothness [30]. $\mathcal{E}$ is the key reconciliation scheme used in [28]. The error reconciliation scheme used in [29] contains a conciliate function $\text{Con}(\cdot)$ and a reconcile function $\text{Rec}(\cdot)$.

acts as the OT sender and picks a pair of random strings as input. The client parses the OT output as the session key. Compared to protocols that use the “projective hashing” paradigm (e.g., [30, 40, 68, 105]), the protocol of [50] is secure against adaptive corruptions without erasures. At EUROCRYPT’18, Dupont et al. [70] proposed a typo-tolerant symmetric PAKE protocol, which also uses the password as input to the OT protocol. Later, Erwig et al. [69] proposed a typo-tolerant asymmetric PAKE protocol. The password file in [69] contains a codeword and random values. The exact position of the codeword is dictated by the password.

- *Password as the private key of the trusted third party.* At CRYPTO’22, Cremers et al. [71] proposed two identity-based symmetric PAKE protocols: CHIP and CRISP. The construction of CHIP is nearly identical to Fiore-Gennaro’s [106] identity-based key exchange protocol, except that the key generated by the trusted third party is replaced by the hash of the password. Compared to CHIP, CRISP can resist pre-computation attacks (see Sec. 2.3.1).

4 FURTHER CLASSIFICATION OF PAKE

Besides the design strategy, there are many other classification methods for PAKE, and here we focus on the five most important ones: symmetry, number of participants, hardness assumptions, security goals, and round complexity. Each classification is described as follows:

- According to symmetry, PAKE protocols can be divided into symmetric PAKE protocols (e.g., [16, 20, 36, 100]) and asymmetric PAKE protocols (e.g., [8, 11, 21, 95]).
- According to the number of participants, PAKE protocols can be divided into two-party (e.g., [8, 11]), three-party (e.g., [24, 76]), and multi-party (e.g., [33, 107]) PAKE protocols.
- According to the hardness assumption, PAKE protocols can be divided into discrete logarithm based (e.g., [16, 20]), RSA-based (e.g., [73, 74]), and lattice-based (e.g., [28, 29]) PAKE protocols.
- According to security goals, PAKE protocols can be divided into forward-secure PAKE (e.g., [14, 57]), provably secure PAKE (e.g., [7, 8]), anonymous PAKE (e.g., [26, 27]), and so on.
- According to the number of rounds, PAKE protocols can be divided into one-round PAKE (e.g., [36, 92]), two-round PAKE (e.g., [58, 64]), three-round PAKE (e.g., [11, 95]), and so on.

4.1 Symmetric and Asymmetric PAKE

To help gain a better understanding, here we give a general definition. *Symmetric PAKE* is a PAKE where both parties play the same roles (e.g., [15, 16, 36, 95]). *Strong symmetric PAKE (sPAKE)* is a symmetric PAKE protocol that is resistant to pre-computation attacks (e.g., CRISP [8]). *Asymmetric PAKE (aPAKE)* is designed to maintain security despite a server compromise (e.g., [10, 11, 21,

34]). *Strong asymmetric PAKE (saPAKE)* is an asymmetric PAKE protocol that is resistant to pre-computation attacks (e.g., OPAQUE [7] and saPAKE-BB [58]).

4.1.1 Symmetric PAKE protocol. Bellare and Merritt [13] for the first time put forward a symmetric PAKE, named EKE. After that, many famous symmetric PAKE protocols have been proposed, such as SPEKE [15], SPAKE2 [36], and J-PAKE [16]. Most symmetric PAKE protocols (except CRISP [8]) assume that both participants hold the password in the clear. The disadvantages of such protocols (e.g., [13, 15, 36]) are inherent. That is, an attacker who corrupts one party can perform key compromise impersonation attacks [7, 37]. Recently, Cremers et al. [71] proposed two symmetric PAKE protocols (i.e., CHIP and CRISP) that do not store a plaintext password, but store a password file with the same structure. Although CHIP and CRISP are symmetric PAKE protocols, they provide security upon server compromise. In particular, CRISP resists pre-computation attacks, which is the highest security that asymmetric PAKE protocols (e.g., [7, 58]) can provide.

Moreover, motivated by the advantages of asymmetric PAKE, Gentry et al. [79] proposed a method (called the Ω -method) to convert any Universal Composable (UC) secure symmetric PAKE protocol to an asymmetric PAKE protocol by adding two additional message rounds and one signature computation/verification. Then, Hwang et al. [9] reduced it to one additional message round. Later, Hesse [114] presented the formal UC-security proof of the Ω -method.

4.1.2 Asymmetric PAKE protocol. Asymmetric PAKE (aPAKE) protocols work in the more common client-server setting, where the server stores a mapping of the password [37, 79]. After EKE [13] was published, Bellare and Merritt [37] soon proposed its asymmetric version, named A-EKE, to prevent direct exposure of passwords upon server compromise. However, if the mapping is deterministic (e.g., a hash of the password), attackers can compute them offline in advance, and then find passwords almost instantaneously when breaking into the server.

To address this problem, Jareki et al. [7] introduced a *strong* asymmetric PAKE notion and provided a general construction (i.e., OPAQUE). Later, Cremers et al. [71] further summarized four methods to construct a password file in asymmetric PAKE protocols. We find that there are at least six types of password files: (1) Hashed password, e.g., A-EKE [37]. (2) Hashed password with identities, e.g., SNAPI-X [74]. (3) Hashed password with a public salt, e.g., VTBEKE [49]. (4) Hashed password with identities and a public salt, e.g., SRP [15]. (5) Hashed password with a secret salt, e.g., saPAKE-BB [58]. (6) Hashed password with identities and a secret salt (no examples). A plausible reason why Cremers et al. [71] only summarized four types is that, if a secret salt is used, the public identifier does not affect the resistance to pre-computation attacks.

- *Hashed password.* A one-way function of the password is stored in the password file, e.g., A-EKE [37], B-SPEKE [87], aEKE-HMQV [8]. Hashing passwords is the simplest way to transform protocols from symmetric to asymmetric, but it cannot resist pre-computation attacks. That is, an attacker can pre-compute the hash value of each password in a password dictionary, and prepare a reverse lookup table from the hash value to the plaintext password. The reverse lookup table works like a rainbow table. Once the password file is compromised, an attacker can recover multiple passwords in $O(1)$ time.
- *Hashed password with public identifiers.* A one-way function of the password with some public identifiers of the participants is computed and stored in the password file, e.g., PAK-X [34], SNAPI-X [74], SPAKE2+ [94]. The public identifiers can be derived from the network name, version, and type, the server name, and the user name. In this case, due to the difference in public identifiers, the hashed value computed by the attacker can't amortize to all targets. But the pre-computation attack still applies to specific entities.

- *Hashed password with public salt.* A one-way function of the password with a random salt is computed and stored in the password file, e.g., SRP [10], VTBPKE [49], AucPace [90]. “Public” means that the salt is sent to the client in the clear, so it can be eavesdropped or intercepted by attackers. An attacker who obtains a public salt $salt$ can pre-compute $H(pw' || salt)$, where $H(\cdot)$ is a hash function and pw, pw' represent the correct password and the attacker’s guess, respectively. Then the attacker can learn all users’ passwords instantly upon server compromise by checking whether $H(pw' || salt) = H(pw || salt)$.
- *Hashed password with secret salt.* A one-way function of the password with a secret random salt is computed and stored in the password file, e.g., OPAQUE [7], saPAKE-BB [58]. The notion of *secret* doesn’t mean that the password must be stored in encrypted form in the password file. The security goal is to prevent attackers from collecting enough data before the password file is compromised. For example, OPAQUE [7] combines the HMQV elements and Oblivious Pseudo-Random Functions (OPRF) to construct a password file. In this case, since the attacker can’t obtain the OPRF key, pre-computation is prevented.

4.2 Two-party, Three-party, and Mutli-party PAKE

4.2.1 Two-party PAKE protocol. The number of two-party PAKE protocols is the largest (over 70) in the literature. Therefore, it is difficult to analyze all relevant typical protocols based solely on the concept of “two-party”. The research focus within two-party PAKE is diverse. For example, as we discussed in Sec. 4.1, researchers focus on the impact of the eventual failure of the server and design symmetric, asymmetric, strong symmetric, and strong asymmetric PAKE protocols. There are other examples: Katz et al. [57] and Becerra et al. [14] focus on satisfying forward secrecy; Gentry et al. [97] and Abdalla et al. [115] focus on providing provable security; Hwang et al. [9] and Katz-Vailuntanathan [30] focus on optimizing protocol rounds. At EUROCRYPT’22, Cramer et al. [109] presented two latest two-party PAKE protocols (i.e., aEKE-HMQV and OKAPE-HMQV) with minimal computational cost and minimal round complexity. In short, the field of two-party PAKE is highly active, with numerous outstanding works being developed.

4.2.2 Three-party PAKE protocol. Three-party PAKE protocols are prompted by the need for practical applications. As stated above, two-party PAKE (2PAKE) protocols are well-suitable for the standard client-server setting. But in large-scale network communication environments, the 2PAKE model is no longer applicable. The requirement for each client to remember the passwords of all its communicating parties seriously affects the storage capacity and scalability of the client. To address this issue, three-party PAKE (3PAKE) protocols are developed as a natural and effective solution. A trusted third party (which acts as the server) is introduced to centrally keep the passwords for all users in 3PAKE protocols. Thus, each user only needs to share one password with the server. In addition to traditional two-user settings, the 3PAKE protocol can also be applied in the wireless sensor setting and the two-server setting. In particular, 3PAKE protocols suffer from the threat of password guessing attacks as well, which can be observed from the “break-fix-break-fix” route of the following protocols, as partially illustrated in the bottom right corner of Fig. 1.

In 1995, Steiner et al. [76] proposed a three-party PAKE for the first time. The PAKE of [76] is an extension of EKE [13]. Abdalla et al. [24] proposed a security model of 3PAKE. Then, Lu and Cao [116] extended the SPAKE2 [36] into a three-party setting without the requirement for any server’s public key. Later, Huang [117] proposed a new 3PAKE without the server’s public key, which is claimed to be more efficient than Lu-Cao’s PAKE [116]. Then Yoon and Yoo [118] pointed out that Huang’s 3PAKE [117] is vulnerable to undetectable online guessing attacks and offline guessing attacks. Subsequently, Tallapally [119] performed an unknown key share attack on Huang’s 3PAKE [117], and then proposed an alternative protocol with four message transmission rounds, which

was breached by Farash and Attari [120]. Lu et al. [121] presented an improved 3PAKE to erase the security flaws found in [120]. However, Chen et al. [122] found that the 3PAKE proposed in [121] is still vulnerable to offline password attacks. Thus, a fundamental challenge with the PAKE is to resist offline guessing attacks, regardless of being two- or three-party.

4.2.3 Multi-party PAKE protocol. Multi-party PAKE is also known as the group password-authenticated key exchange (GPAKE). Distributed group communication applications, such as remote teaching and multimedia conferencing, motivate the need for secure and practicable multi-party authentication protocols. Generally, GPAKE was originally introduced as the extension of two-party PAKE. However, large-scale communication and unreliable data transmission put forward more requirements on the GPAKE. On the one hand, considering the uncertainty of the number of participants in the application scenario, GPAKE should achieve constant rounds [123]. Namely, the number of communication rounds is independent of the cardinality of group members, so it will not increase linearly or logarithmically. On the other hand, GPAKE should support dynamic updates of members. In group communication, newly joined members should not have access to previous conversation messages, and quit members should not continue to receive new messages.

Moreover, GPAKE protocols proposed in the literature can be mainly divided into two categories according to whether there is a trusted third party. In the GPAKE protocol with a trusted third party (also called a central server) in the system, each client shares an independent password with the server. Then, when users wish to create a session key, they resort to a trusted third party for authenticating each other. For example, an authentication server was added in the GPAKE protocol proposed by Abdalla and Pointcheval [107]. This protocol combines the Completely Automated Public Turing Test to Tell Computers and Humans Apart (CAPTCHA) technique and federated signature verification to thwart undetectable online password guessing attacks. In contrast, in the GPAKE protocol without any trusted third party, all participants achieve authentication in a self-organized manner and collaborate to calculate the session key. A typical GPAKE protocol without a trusted server was proposed by Wan et al. [33]. In order to achieve higher computational efficiency, they combined the basic Diffie-Hellman exchange and the hierarchical key tree structure.

4.3 Hardness Assumptions of PAKE

4.3.1 PAKE based on the discrete logarithm problem. Diffie and Hellman [86] for the first time proposed a one-way trapdoor function with excellent asymmetry: modular exponentiation, and used it to demonstrate the famous cryptographic protocol: the Diffie-Hellman key exchange protocol. The classical Diffie-Hellman key exchange provides no authentication at all, and only focuses on the establishment of the session key. The confidentiality of the shared key in [86] is guaranteed by the Computational Diffie-Hellman (CDH) problem. Then Miller [124] and Koblitz [125] introduced the elliptic curves over finite field, respectively. ElGamal [126] further proposed a public key cryptosystem. Notably, elliptic-curve cryptography is excellent in key length. Under the same security strength, its key strength is much smaller than textcolorblue that of the integer-factorization based cryptography, implying smaller bandwidth and storage requirements [127].

The seminal work that applied the above problems to password authentication protocols is Bellovin and Merritt [13]. After that, researchers have been committed to basing the security of the PAKE protocol on the intractability of solving the discrete logarithm (DL) problem. In addition to the wide application in PAKE protocols, these hardness problems are also common in two-factor or multi-factor authentication protocols. On the other hand, the Diffie-Hellman key exchange technique is one of the promising methods to achieve perfect forward secrecy [128], which makes it more attractive to researchers. Some classic protocols (e.g., [15, 16, 21, 38]) have also been included in international standards ISO/IEC 11770-4 [89]. Among these standardized

PAKE protocols, AMP [38], J-PAKE [16] and P-SPEKE [20] work in both the DL and EC settings, but SRP-6 [21] only works in the DL setting. The underlying reason is that the mathematical expression $Y^* = g^y + 3 \cdot g^{H(ID_U \| pw \| salt)}$ used in SRP-6 (see Table 6) cannot be defined in the EC setting.

4.3.2 PAKE based on the integer factorization problem. The best-known integer factorization (IF) problem is the RSA, named after its inventors Rivest, Shamir and Adleman [129]. The security of RSA lies in the difficulty of computing the e -th root of a ciphertext c ($c = m^e \bmod N$) module a composite integer N . As a relatively mature public key cryptosystem, RSA has been employed in password-authenticated key exchange as early as the seminal work of Bellare and Merritt [13]. The basic idea of constructing a PAKE protocol based on RSA consists of three steps. First, the sender generates and sends the RSA public key (n, e) . Then the receiver uses the public key to encrypt the key material with the shared password as a parameter. According to the RSA algorithm, the party accepting the public key (n, e) should check the validity of the equation $\gcd(e, \varphi(n)) = 1$.

The purpose of checking the equality is to ensure that no matter whether the password parameter is the correct password, the ciphertext encrypted by RSA remains an e -residue. Thus, an attacker cannot rule out wrong passwords in one attempt. Authors generally take other measures to avoid a direct primality test, such as requiring exponent e to be greater than modulus n [74], or using multiple RSA encryption [35], thereby reducing the computational cost (see Sec. 3.1.2). The final phase is decryption, and key confirmation can be attached to achieve explicit authentication. As shown in Table 2, few RSA-based PAKE protocols have been proposed recently, and similar situations occur in two-factor and multi-factor password authentication. A reasonable explanation can be given in terms of security and usability: (1) The RSA-based PAKE protocol is vulnerable to e -residue attacks and does poorly in forward secrecy (see Table 9). By contrast, these security issues are rare in DL-based PAKE protocols. (2) Massive modular exponentiation and multiplication inverse make RSA-based PAKE protocols insufficiently efficient for practical use.

4.3.3 PAKE based on the lattice problem. Regev [130] reduced the worst-case lattice problems (e.g., GAPSVP and SIVP) to the LWE problem for the first time, and presented a classical public-key cryptosystem whose security is based on the hardness of the Learning With Errors (LWE) problem. Compared to traditional DL problem and IF problem, the hard problems over lattices can resist quantum computing attacks [32]. The quantum computer has faster parallel computing and simulation capabilities, so they can solve (elliptic curve) discrete logarithm and integer factorization problems in polynomial time [131]. In contrast, the lattice problem is a mathematically hard problem that cannot be solved by a quantum computer, so it can provide security in the context of quantum-safe cryptography. The lattice-based PAKE protocol is currently considered to be the most promising research direction of quantum-resistant cryptographic protocols.

Interestingly, a research route shows the pursuit of the optimal round of the PAKE protocol over lattices. Specifically, at ASIACRYPT'09, Katz and Vaikuntanathan [68] introduced a generic *three*-round symmetric PAKE protocol over lattices by a public-key encryption scheme and SPHF. Then Zhang and Yu [31] proposed a *two*-round symmetric PAKE based on the hardness of the LWE assumption. later, Li et al. [32] proposed a *one*-round asymmetric PAKE over lattices, eliminating Non-Interactive Zero-Knowledge (NIZK) and assumptions of the random oracle model.

4.4 Security Goals of PAKE

4.4.1 Forward-secure PAKE protocol. Forward secrecy is a desirable security property of PAKE protocols, implying the independence of the session key on the long-term key. Forward secrecy can be distinguished as Perfect Forward Secrecy (PFS) and weak Forward Secrecy (wFS) according to whether an adversary actively participates. The commonly referred to forward secrecy, or perfect forward secrecy, means that even if the long-term private key of one or more parties is stolen by an

attacker, the security of the session key established in the previous session is still not threatened [44]. But weak forward secrecy only protects the session key created without the active participation of attackers after the long-term private key material is compromised [39]. Recall that the capability of C-6 in Table 1 captures the leakage of long-term private keys, corresponding to forward secrecy here. Also, corrupt oracles are introduced into the security model to simulate it (see Sec. 2.2).

The notion of forward secrecy was first proposed by Diffie [132], and was formally applied to password authentication protocols in [43] and [57]. Then, Park et al. [128] further proposed two methods to achieve forward secrecy in the PAKE protocol. One method is based on the classic Diffie-Hellman key exchange technique, and the other is based on the confidentiality of a random number selected by the server. Similarly, Ma et al. [133] put forward a general principle, namely that at least two exponentiations (respectively point multiplication in elliptic curve cryptosystems) operations conducted on the server side are necessary for achieving forward secrecy. In addition to basic cryptographic components, some researchers have focused on the relationship between protocol flow and forward secrecy. For example, Krawczyk [39] pointed out that a two-flow PAKE protocol using a public-key primitive as an authentication mechanism can not satisfy perfect forward secrecy, and proposed the notion of weak forward secrecy (wFS). Following this idea, Becerra et al. [14] proved that the SPAKE2 protocol [36] only satisfies wFS, and proposed the PFS-SPAKE2 protocol by incorporating key-confirmation codes into the original SPAKE2 protocol. Recently, forward secrecy has become a basic goal of security protocol design. For example, all public-key based key exchange mechanisms in TLS 1.3 have provided this property [134].

4.4.2 Provably secure PAKE protocol. When a password authentication protocol is proposed, authors usually present proof of security to ensure the validity and practicability of the PAKE protocol (e.g., [7, 8, 58]). Most PAKE protocols published earlier performed only heuristic and informal security cryptanalysis. As time goes on, more and more protocols are analyzed with a formal method [135], such as complexity-based provable security [43], symbol-based logic [20], and automated formal verification tools [136]. Among them, complexity-based provable security in cryptography refers to reducing breaking the protocol to solving hard problems (e.g., CDH, RSA) or cryptographic primitives (e.g., the discrete logarithm problem, the integer factorization problem) [137]. Because the advantage of computers in solving these problems in polynomial time is negligible, the possibility of an attacker cracking the PAKE protocol is also negligible.

A prerequisite for a PAKE protocol to be provably secure is a formal, well-defined security model, such as the *random oracle model* and the *standard model*. The provable security of the early protocol (e.g., KOY [91]) is mainly based on the standard model, where the simulation of the protocol is realized by constructing a special trapdoor function without random oracle assumptions. Conversely, the random oracle model assumes that some cryptographic hash functions are replaced by a publicly accessible random function (i.e., a random oracle). An adversary can only obtain the result of the hash function by querying the random oracle, instead of by calculating. In addition, the quantum random oracle is proposed to resist quantum computer attacks. The random oracle model requires that the emulator can record and query all queries executed by adversaries. But in the context of quantum-safe cryptography, the adversary's query cannot be recorded and read, resulting in the ordinary random oracle being unable to work. To overcome this issue, researchers have proposed multiple methods (see [138, 139]) to build a scientific quantum random oracle model.

Moreover, the *ideal cipher model (ICM)* has also appeared in the development of the security model. Instead of having a publicly accessible random function, it assumes that there is a publicly accessible random block cipher (or ideal cipher). For any given key, all parties including an adversary can simultaneously perform encryption or decryption queries on the ideal block cipher. Coron et

Table 8. 23 PAKE Protocols with minimal round complexity (i.e., implicit authentication in 1-round/2-flow; unilaterally explicit authentication in 2-round/2-flow; mutually explicit authentication in 3-round/3-flow). The PAKE protocol with the best overall security and performance in each round is highlighted in **purple**.

Protocol	Year	Reference	R/F	Start	EA	Symm.	Client	Server	Assump.	Model
aEKE-HMQV	2022	EUROCRYPT [8]	1/2	-	None	aPAKE	1fb+1mbv	1fb+1mbv	gapDH	RO/IC
McQuoid et al.	2020	ACM CCS [100]	1/2	-	None	PAKE	1fb+1vb	1fb+1vb	DDH	RO
P-SPEKE	2018	IEEE TIFS [20]	1/2	-	None	PAKE	2vb	2vb	CDH	-
SPAKE2	2005	CT-RSA [36]	1/2	-	None	PAKE	3fb+1vb	3fb+1vb	CDH	RO
SPAKE1	2005	CT-RSA [36]	1/2	-	None	PAKE	3fb+1vb	3fb+1vb	CDH	RO
Kobara-Imai	2002	IEICE Trans. [92]	1/2	-	None	PAKE	2fb+1vb	2fb+1vb	DDH	Std
PPK	2000	EUROCRYPT [34]	1/2	-	None	PAKE	1fb+2vb	1fb+2vb	DDH	RO
OKAPE-HMQV	2022	EUROCRYPT [8]	2/2	S	C	aPAKE	1fb+1mbv	1fb+1mbv	gapDH	RO/IC
saPAKE-BB	2019	CRYPTO [58]	2/2	C	S	saPAKE	10fb+1bv	2fb+2vb	SDH, DDH	RO
OMDHKE	2004	PKC [64]	2/2	C	S	PAKE	1fb+1vb	1fb+1vb	CDH	RO
AuthA	2000	IEEE P1363 [72]	2/2	C	S	aPAKE	1fb+1vb	1f+1vb	CDH	RO/IC
KHAPE	2021	CRYPTO [11]	3/3	S	C+S	aPAKE	1fb+1mbv	1fb+1mbv	gapCDH	IC
KC-SPAKE2+	2020	TCC [95]	3/3	C	C+S	aPAKE	2fb+2vb	2fb+2vb	CDH	RO
KC-SPAKE2	2020	TCC [95]	3/3	C	C+S	PAKE	2fb+1vb	2fb+1vb	CDH	RO
OPAQUE	2018	EUROCRYPT [7]	3/3	C	C+S	saPAKE	1fb+2vb+1mbv	1fb+1vb+1mbv	OMDH	RO
PFS-SPAKE2	2018	ProvSec [14]	3/3	C	C+S	PAKE	2fb+1vb	2fb+1vb	CDH	RO
PAK-Z+	2005	IEEE P1363 [97]	3/3	C	C+S	aPAKE	1fb+1vb	1fb+1vb	CDH	RO
PAK-Z	2002	IEEE P1363 [67]	3/3	C	C+S	aPAKE	1fb+1vb	1fb+1vb	CDH	RO
PAK-RY	2001	CT-RSA [66]	3/3	C	C+S	aPAKE	2fb+2vb	1fb+3vb	CDH	RO
PAK-R	2001	CT-RSA [66]	3/3	C	C+S	PAKE	1fb+2vb	1fb+1vb	DDH	RO
PAK-X	2000	EUROCRYPT [34]	3/3	C	C+S	aPAKE	2fb+3vb	2fb+3vb	DDH	RO
PAK	2000	EUROCRYPT [34]	3/3	C	C+S	PAKE	1fb+2vb	1fb+2vb	DDH	-
M-EKE	1995	ACM SIGOPS [76]	3/3	C	C+S	PAKE	1fb+1vb	1fb+1vb	CDH	RO/IC
R/F: Round/Flow			EA: Explicit Authenticaiton			SDH: Strong Diffie-Hellman			C, S: Client, Server	
IC: Ideal Cipher			fb: Fixed-based Exponentiation			CDH: Computational Diffie-Hellman			gapDH: gap Diffie-Hellman	
RO: Random Oracle			vb: Variable-based Exponentiation			DDH: Decision Diffie-Hellman			aPAKE: Asymmetric PAKE	
Std: Standard Model			mbv: Multi-exponentiation			OMDH: One-More Diffie-Hellman			saPAKE: Strong asymmetric PAKE	

al. [140] further proved that the random oracle model and the ideal cipher model are inherently equivalent. Table 9 shows the security model adopted by representative PAKE protocols.

However, the traditional provable security theory ignores the security when multiple protocols are executed in combination. Canetti et al. [141] addressed this issue by proposing a new paradigm for defining the security of PAKE protocols, called *universally composable (UC)* security. This framework requires security to be assured even when the cryptographic protocol is composed of any set of protocols. Commonly, the UC framework is combined with assumptions such as random oracles to provide rigorous security guarantees for a PAKE protocol. Recently, Abdalla et al. [142] introduced the lazy-extraction UC PAKE and relaxed UC PAKE functionalities, and showed several PAKE protocols (e.g., SPAKE2 [36], TBPEKE [49] and SPEKE [15]) that are previously proven secure with respect to a game-based definition can realize the lazy-extraction UC PAKE functionality.

4.4.3 Anonymous PAKE protocol. Anonymity was not considered in early PAKE protocols, and all participants were publicly identifiable (e.g., [15, 34, 37]). Concretely, the identifiers are sent in the clear, as part of the PAKE protocol. With the increasing awareness of privacy in society, anonymous PAKE protocols that can protect data access information are receiving more and more attention. The anonymous PAKE protocol provides identity authentication, key establishment, and anonymity at the same time. From the perspective of participants, anonymity can be divided into user anonymity and server anonymity. User anonymity means that the protocol protects the user's identity and achieves the untraceability of user behavior [44]. Specifically, an attacker can neither confirm the identity of the user nor distinguish whether any two sessions are from the same user. Server anonymity means that the password database information will not be disclosed to users. Except whether the user password is stored in the server database, no information about the

password or the set of valid passwords should leak to the other party [26]. Note that, in addition to anonymous PAKE protocols, achieving any form of network anonymous communication inherently requires the encryption of data packets to hide the source and destination.

Generally, most anonymous PAKE protocols either require the client to remember an index assigned to its password in the server's password database (e.g., [143, 144]), or require the client to store password-wrapped anonymous credentials (e.g., [27, 41, 42]). Considering the storage requirement of the client, Vasco et al. [26] proposed two anonymous PAKE protocols that assume the client holds only passwords. The first protocol leverages oblivious pseudo-random functions, and the second one builds upon a special type of identity-based encryption scheme. Vasco et al.'s work [26] breaks the constraints of the previous two types, and truly enables the construction of anonymous PAKE protocols using only passwords. In addition, public-key techniques are not only essential for constructing password authentication protocols that resist offline password guessing attacks [145], but also essentially indispensable to constructing an anonymous PAKE protocol [146]. Building an anonymous PAKE protocol using only basic symmetric cryptographic primitives (e.g., hash functions, symmetric encryptions, and XOR operations) is extremely unlikely.

4.5 Round Complexity of PAKE

The primary criteria for evaluating a PAKE are round complexity, computation overhead, and security. The research trend on PAKE is also oriented toward these three criteria. For example, the saPAKE-BB protocol [58] achieves strong asymmetric security as OPAQUE [7], and further reduces the computational cost and round complexity; The aEKE-HMQV protocol [71] achieves asymmetric security and computational cost as KHAPE [11], and further reduces round complexity (see Fig. 3). Table 8 summarizes 23 PAKE protocols with the minimum round complexity.

Minimal round complexity. The number of rounds and implicit/explicit authentication are related. The PAKE with one-round communication can only achieve implicit authentication (e.g., SPAKE2 [94]). There are two methods to reduce round complexity: 1) merge messages from different flows, e.g., M-EKE [76]; 2) turn two ordered flows into unordered ones, e.g., P-SPEKE [20]. Specifically, the M-EKE [76] is essentially the same as EKE [13], except that EKE has five rounds of communication and M-EKE has three rounds of communication. M-EKE [76] merges the fourth message of EKE (i.e., the key confirmation code from the server) into the second message, and the fifth message (i.e., the key confirmation code from the client) into the third message, thereby reducing the number of rounds. The P-SPEKE [20] only has two rounds (four flows): the first round is used for key exchange, and the second round is used for key confirmation. The messages sent by the client and server are completely symmetrical, and one doesn't need to wait for the other party to send its own message.

Minimal computation overhead. Generally, unauthenticated key exchange (KE), i.e., Diffie-Hellman, is used as a lower-bound for symmetric PAKE and asymmetric PAKE (aPAKE), namely, aPAKE $\Rightarrow$ PAKE $\Rightarrow$ KE [8, 11]. The computational cost of the Diffie-Hellman key exchange is one offline fixed-base and one variable-base exponentiation per party. Note that the computation costs in Table 8 only involve exponentiation operations and do not include hashing, encryption, etc.

The computational cost of the five symmetric PAKE protocols (i.e., [20, 43, 64, 76, 100]) matches Diffie-Hellman key exchange. More specifically, AuthA [72] and M-EKE [76] employ password-encryption to Diffie-Hellman key exchange messages (see Table 2). OMDHKE [64] and McQuoid et al. [100] apply a mask generation function as the product of a Diffie-Hellman value with a hash of the password (see Table 5). P-SPEKE [20] applies a hash function, which uses the password to derive a secret group generator for performing Diffie-Hellman key exchange (see Table 3). The three asymmetric PAKE protocols (i.e., [8, 11]) have close to the optimal computational cost of one offline fixed-base exponentiation and one multi-exponentiation (at the cost of 1.17 regular

	OPAQUE	saPAKE-BB	KHAPE	aEKE-HMQV
	2018	2019	2021	2022
Strong aPAKE security	saPAKE	saPAKE	aPAKE	aPAKE
Computation (Client-side)	1fb+2vb+1mbv	10fb+1vb	1fb+1mbv	1fb+1mbv
Round (Client initiates)	3-round (C+S)	3-round (C+S)	4-round (C+S)	2-round (C+S)

Fig. 3. An example that illustrates the relationship between security, computation overhead, and round complexity. The four PAKE protocols are OPAQUE [7], saPAKE-BB [58], KHAPE [11] and aEKE-HMQV [8]. For a fair comparison, we give the number of rounds required for each protocol to implement mutually explicit authentication. The value highlighted in **red** indicates the improvement of the current protocol over the previous one. fb = fixed-based exponentiation, where the base is fixed and precomputation techniques apply to accelerate computation (e.g., computing g^x for a fixed generator g); vb = variable-based exponentiation, where the base is determined at runtime and cannot be precomputed (e.g., computing g^x with $g = H(pw)$); mbv = multi-exponentiation, referring to computing multiple exponentiations (e.g., computing $g^x h^y$).

exponentiations) per party. More specifically, KHAPE [11], aEKE-HMQV [8], and OKAPE-HMQV [8] use (the hash value of) the password to the public key and/or private key of HMQV.

Strong aPAKE security. At EUROCRYPT'18, Jarecki et al. [7] strengthened the definition of aPAKE to eliminate the vulnerability to pre-computation attacks (i.e., strong aPAKE security). OPAQUE [7] offers resistance to pre-computation attacks by employing oblivious pseudo-random functions (OPRF); saPAKE-BB [58] employs a new form of conditional key encapsulation mechanism (CKEM). In a word, a strong aPAKE must hide the salt, so that $\mathcal{A}$ cannot make any meaningful pre-computation before a compromise. For more information on strong aPAKE security, see Sec. 4.1.2.

To gain a better insight into the subtleties between security, computational cost, and round complexity, we use the latest four protocols as examples (see Fig. 3). The most significant feature of OPAQUE [7] is its strong aPAKE security. However, the computational cost of OPAQUE is greater than that of unauthenticated key exchange. This is because OPAQUE uses both OPRF (1fb+1vb) and HMQV (1fb+1mbv). By employing CKEM, saPAKE-BB [58] uses more fixed-base exponentiation operations, thereby reducing the computational cost. KHAPE [11] improves performance by dispensing with the OPRF (hence also reducing security). Recently, aEKE-HMQV [8] achieves both minimal computational cost and minimal round complexity by using *one-pass* HMQV. In standard HMQV, $(a, A = g^a)$ and $(b, B = g^b)$ are long-term private-public key pairs for the client and server, respectively. But in one-pass HMQV, (b, B) plays a double role as a server's public key and its ephemeral Diffie-Hellman contribution. Therefore, even if an attacker decrypts B' using a candidate password pw' from a password dictionary, she cannot perform offline password guessing attacks. Because B is random every session and cannot be used by the attacker to rule out wrong passwords. As a result, the messages of users and servers in aEKE-HMQV [8] are synchronized.

5 COMPARISON

To demonstrate the effectiveness of our framework in practice, we provide a comparative evaluation of 71 representative PAKE protocols in terms of security, usability, and efficiency. This assessment system is based on the security model proposed in Sec. 2.2 and 13 desirable properties proposed in Sec. 2.3. Table 9 offers a comparison of 71 PAKE protocols in terms of security and usability. Table 10 shows the efficiency comparison of 71 PAKE protocols. Note that the survey conducted in this paper is not absolutely complete. A PAKE protocol is included in Tables 9 and 10 only because it is highly representative, or clarifies a certain design strategy in some way (see Sec. 3).

To select these 71 PAKE protocols, we proceed from two perspectives. First, we start from a list of seed papers and search improvement schemes proposed by researchers for these PAKE protocols.

Seed papers are representative works that we know related to password authentication, e.g., the seminal EKE protocol [13], the SRP protocol [10], and the SPEKE protocol [15]. Most of them are widely recognized in the cryptographic community, and have even been incorporated into the international standard. Then, we trace back from excellent works in the past three years, such as the CRISP protocol [71] that is resilient to pre-computation attacks, and Vasco et al.'s [26] anonymous protocol that is truly password-only. We believe that recent works indicate the hot issues in PAKE protocols research and suggest future directions. To identify more works, we further examine PAKE protocols with special attributes from the perspective of protocol classification as discussed in Sec. 4. These carefully chosen PAKE protocols are sorted by publication date, creating a timeline for PAKE protocol development. Furthermore, papers are identified by the name of the proposed PAKE protocol if available (e.g., SRP [10]), otherwise by the authors' name (e.g., Vasco et al. [26]). These 71 works are published in 59 venues. The top-3 venues for publishing papers on PAKE protocols are EUROCRYPT (8 papers), ACM CCS (7), and ASIACRYPT (5).

5.1 Comparison of Security and Usability

There are four strong asymmetric PAKEs (i.e., saPAKE-BB [58], OPAQUE [7], DE-PAKE [65] and Vasco et al. [26]) and one strong symmetric PAKE (i.e., CRISP [71]). Recall that the "strong" notion disallows pre-computation attacks [7]. Due to frequent leakage of password database [149–151], the security of password-based authentication is questioned. Because, after a server compromise, an attacker can find users' passwords through offline guessing attacks. But the proposal of strong (a)PAKE may reduce this concern. Strong (a)PAKE prevents the security of passwords from being *immediately* broken, and any meaningful pre-computation is infeasible before a server compromise. As another interesting discovery, researchers tend to analyze PAKE protocols with complexity-based provable security, rather than symbol-based logic and automated formal verification tools.

In addition, there is no PAKE protocol in Table 9 that does not satisfy wFS (or PFS). This is because public-key cryptology is used. For example, in EKE [13], after the password pw is leaked, an attacker can decrypt g^x and g^y from $E_{pw}(g^x)$ and $E_{pw}(g^y)$, but cannot compute g^{xy} , so EKE at least satisfies wFS. Besides, four anonymous PAKEs (i.e., [26, 27, 41, 42]) are listed in Table 9.

Our analysis of Table 9 follows the order from left to right, that is, from "Security model" to "U7 Round complexity". From the perspective of the security model, the use of universally composable (UC) frameworks increases over time: UC PAKE accounts for 12/20 in the past five years. For example, Erwig et al. [69] presents a security definition for asymmetric fuzzy PAKE in the UC model; Jarecki et al. [7] formalizes strong asymmetric PAKE in the UC model; Gu et al. [11] presents a formalization of the key-hiding AKE UC functionality. In addition to these new security definitions, the interaction with the environment in the UC model can capture the actual deployment of PAKE. Because the entire network activity consists of many executions, PAKE is just one of them (e.g., OPAQUE can be used in conjunction with TLS. For details on their combination, see [7]). This indicates that researchers pay more attention to the security of the protocol combination execution.

Diffie-Hellman (DH)-like assumptions (e.g., CDH, DDH, and SDH) are the security foundation of most (55/71=77.4%) PAKE protocols. One possible reason why the DH assumption stands out from other cryptography assumptions is its simplicity (only one round of communication) and efficiency (only one offline fixed-base exponentiation and one online variable-based exponentiation per party). In addition, one can observe that lattice-based PAKE shows an upward trend, while RSA-based PAKE is decreasing. As we discussed above, the interest in lattice-based PAKE protocol is largely due to its post-quantum security properties. RSA-based PAKE (e.g., [35, 73, 74]) performs poorly in terms of transmission overhead, round complexity, and support for pre-computation (see Table 9). *Security analysis.* Among the six security-related criteria, S1 (key agreement) and S2 (mutual authentication) are met by all 71 PAKE protocols. This is because key agreement and mutual

authentication are the inherent nature of a PAKE protocol. There are two main methods of key establishment, namely, key agreement and key distribution. The inherent shortcoming of key distribution is the absolute trust in a single entity, so that if the entity is compromised, the security of the entire system will be destroyed. Hence, PAKE protocols commonly adopt key agreement to generate session keys, which is also more suitable for distributed environments or ad hoc networks. S3 (explicit authentication) and S4 (perfect forward secrecy = PFS) are related. Namely, PFS cannot be achieved by any implicitly authenticated 2-message protocols based on public-key authentication [39]. As shown in Table 9, 17 implicitly authenticated protocols only satisfy weak forward secrecy.

It is good news to see that S6-1 (resistance to replay attacks) and S6-2 (resistance to man-in-the-middle attacks) are achieved by 71 PAKE protocols. A method to prevent replay attacks is to use random numbers (implicitly adopted by all 71 PAKE protocols) or time stamps. The typical victim of man-in-the-middle attacks is the MQV protocol proposed by Menezes et al. [152], which was later improved by Krawczyk [39] into the famous HMQV protocol. An effective solution to resist man-in-the-middle attacks is to add identifiers to the session key, e.g., SPAKE2 [36] and P-SPEKE [20], thus ensuring that the session key is established with the intended entity. Another method is to bind messages to the identifier using zero-knowledge proof techniques, e.g., J-PAKE [16].

In addition, three RSA-based PAKE protocols (i.e., OKE [73], SNAPI [74] and SNAPI-X [74]) are vulnerable to partition attacks, which combine online guessing and offline guessing [85]. Three PAKE protocols (i.e., EKE [13], A-EKE [37], and M-EKE [76]) suffer from number theory attacks, so offline guessing is feasible [23]. SRP [10] is vulnerable to undetectable online guessing attacks [21]. SPEKE [15] and B-SPEKE [87] are vulnerable to unknown key-share attacks and parallel-session attacks [20]. Finally, EKE [13] is vulnerable to a known-key attack. That is, attackers who know the session key in EKE [13] can learn the user's password by performing offline guessing attacks [76].

There are 64.8% (46/71) of PAKE protocols that meet S6-8 (resistance to ephemeral secret leakage). PAKE protocols that fail to meet S6-8 are all due to the absence of long-term secrets (such as passwords [14] or the server's long-term keys [26]) in the session key material. Moreover, when it comes to the compromise of the long-term keys, the security property of resistance to key compromise impersonation (KCI) attacks has been ignored. But it is of practical importance and deserves attention. Considering the cloud-storage scenario, if the long-term private key of the server is compromised, it is unavoidable for the attacker to impersonate the server, but impersonating users to the server (to access cloud files) should be avoided. Table 9 reveals that none of the symmetric PAKE protocols achieve resistance to KCI attacks, owing to the fact that participants all possess plaintext passwords or password files with the same structure. But even in an asymmetric structure, 29.0% (9/31) of protocols fail to provide full resistance to KCI attacks. For example, in the SPAKE2+ protocol [94], a malicious user who has the server's password file can cheat the server to pass authentication, and a server who has an honest user's password can fake the user's login request. This is because SPAKE2+ only requires the user to prove the knowledge of $H(ID_U || ID_S || pw)$ instead of pw , while the value $H(ID_U || ID_S || pw)$ also exists in the password file.

Moreover, there is a subtle link between asymmetry and S6-10 (resistance to pre-computation attacks). That is, (1) symmetric PAKE protocols that only share passwords (e.g., [15, 16]) cannot resist server compromise; (2) strong (a)symmetric PAKE protocols (e.g., [7, 58]) all fully satisfy S6-10; (3) asymmetric PAKE protocols achieve a higher proportion of server compromise resilience. The reason is that the asymmetric PAKE protocol was first proposed in [37] to take into account the possibility of insecure key servers. However, unfortunately, not all asymmetric PAKE protocols can successfully resist server compromise. Of the 31 asymmetric PAKE protocols, eight are unable to resist server compromise (i.e., using deterministic mapping values of passwords), 15 protocols partially resist server compromise (i.e., using mapping values of passwords and identifiers/public random salts), and only eight fully resist server compromise (i.e., using mapping values of passwords

and secret random salts). In addition, only OMDHKE [64] tackles denial-of-service (DoS) attacks. A possible reason is that DoS attacks are more effectively mitigated through defense mechanisms from system security, e.g., throttling and rate limiting (see [153, 154]).

Usability analysis. All 71 PAKE protocols satisfy U1 (password friendly). However, there are three anonymous protocols (i.e., Yang et al. [41], Yang et al. [42] and Zhang et al. [27]) that do not satisfy U2 (login device friendly). This is because the clients of these three protocols need to store password-based credentials. This forces the user to log in on the device where the credential is stored (rather than any device). Almost all PAKE protocols assume that the user only holds a password, which also seems to match the practical situation. However, as an independent natural individual, the user needs software composed of code to execute the protocol algorithm. Therefore, the property of U2 is concerned with whether the client stores parameters related to the user. It is acceptable to store the necessary public parameters, such as group elements and common reference strings, but additional credentials (see [41, 42]) are redundant burdens and should be avoided.

In addition to RSA-based PAKE protocols [35, 73, 74], protocols that convert passwords to generators (e.g., SPEKE [15]) are also prone to not supporting pre-computation. This is because before the user enters the password pw , the client cannot compute the generator $g \leftarrow f(pw)$, and therefore cannot pre-compute g^x (see Table 3). Actually, there may be different benchmarks for achieving the remaining four usability-related criteria (i.e., U4~U7). The specific values of storage, communication, and transmission overhead are shown in Table 10. We just try to provide an intuitive comparison of 71 PAKE protocols in Table 9. Specifically, our benchmarks are as follows.

- A PAKE protocol is said to satisfy U4 (storage overhead) if the storage overhead is less than $3\mathbb{G}$. As with [14], we denote a group element by $\mathbb{G}$ and a κ -bit string by κ .
- A PAKE protocol is said to satisfy U5 (computation overhead) if the computation overhead per party is less than one multi-exponentiation. We ignore fixed-based exponentiations because they can be pre-computed and therefore do not increase the protocol execution time.
- A PAKE protocol is said to satisfy U6 (transmission overhead) if the communication overhead per party is less than $1\mathbb{G}$ for implicit authentication, or less than $1\mathbb{G} + 1\kappa$ for unilaterally/mutually explicit authentication.
- A PAKE protocol is said to satisfy U7 (round complexity) if it can achieve implicit authentication in 1-round/2-flow, unilaterally explicit authentication in 2-round/2-flow, or mutually explicit authentication in 3-round/3-flow.

In all, we can deduce that *no* scheme can fulfill all these 13 criteria – the only five PAKE protocols that can achieve 11 criteria (i.e., except for S5 user anonymity and S6 resistance to known attacks) are KC-SPAKE2+ [95], KC-SPAKE2 [95], PFS-SPAKE2 [20], Kobara-Imai [92], and M-EKE [76].

5.2 Comparison of Efficiency

A total of 14 PAKEs support asynchronous communication: eight protocols with 1-round/2-flow, four protocols with 2-round/4-flow, and two protocols with 3-round/6-flow (see Table 10). We strictly follow the original paper and do not optimize the number of rounds and flows of PAKEs. For example, the key exchange phase of SPEKE [15] is 1-round/2-flow, and the key confirmation phase is 2-round/2-flow, resulting in a total round complexity of 3-round/4-flow. Although the key confirmation phase can be optimized to 1-round, we maintain the original description.

Table 10 only shows exponentiation operations. Hashing, XOR, and encryption are omitted. We adopt the computation, communication, and storage overhead of Katz-Vaikuntanathan [157], Groace-Katz [56] in Jiang et al.'s work [28]. Fixed-base exponentiations (fb) can be pre-computed offline (e.g., $X = g^x$, $Y = g^y$ in SPAKE2 [36]). Variable-based exponentiations (vb) need to be computed online (e.g., X^y , Y^x in SPAKE2 [36]). Generally, PAKE protocols using HMQV (e.g.,

Table 10. Efficiency comparison of 71 representative PAKE protocols.

Protocol	Year	Ref.	R/F	EA	Computation overhead		Communication overhead		Storage overhead	Building blocks
					Client side	Server side	Client side	Server side		
KEM-PAKE	2025	[147]	3/3	C+S	1pke+1sig+2ae	3pke+2ae	1M + 1κ	1M + 1κ	1M + 1κ	KEM, AE
LPKE-PAKE	2024	[148]	3/3	C+S	2pke	2pke	1M + 1κ	2κ	0	LPKE, PKE
BarePAKE	2024	[80]	2/4	C	2fb+2mnb	2fb+2mnb	2G + 2κ	2G	2G + 1κ	Password-enc
SweetPAKE	2024	[81]	3/3	C	(2n+2)fb+2mnb	4nfb+2mnb	2nG + (n+1)κ	2nG + (n+1)κ	n	PKE, PRF, Honeyword
FuzzyPAKE	2023	[82]	2/2	C	2nfb	(2n+2)fb	nG + 1κ	nG	0	ECC, RSS
OKAPE-HMQV	2022	[8]	2/2	C	1fb+1mnb	1fb+1mnb	1G + 1κ	1G	1G + 1κ	Password-enc, HMQV
aEKE-HMQV	2022	[8]	1/2	None	1fb+1mnb	1fb+1mnb	1G	1G	1G + 1κ	Password-enc, HMQV
CRISP	2022	[71]	1/2	None	3vb	3vb	2G	2G	3G	Pairing, PAKE, Htg
CHIP	2022	[71]	1/2	None	1fb+1vb+1mnb	1fb+1vb+1mnb	2G	2G	3G	Pairing, PAKE
KHAPE	2021	[11]	3/3	C+S	1fb+1mnb	1fb+1mnb	1G + 1κ	1G + 2κ	2G + 1κ	Password-enc, HMQV
H-VPAGE	2021	[12]	2/2	None	2vb+4mnb	2vb+4mnb	6G	6G	1G	SPHF, Hardware
Vasco et al.	2021	[26]	5/5	C	1vb	1vb	1G + 1κ	(2n+1)G	(3n+3)G	OPRF
KC-SPAKE2+	2020	[95]	3/3	C+S	2fb+2vb	2fb+2vb	1G + 1κ	1G + 1κ	0	–
KC-SPAKE2	2020	[95]	3/3	C+S	2fb+1vb	2fb+1vb	1G + 1κ	1G + 1κ	0	–
McQuoid et al.	2020	[100]	2/4	C+S	1fb+1vb	1fb+1vb	1G + 3κ	1G + 3κ	0	Hash-to-curve, OT
RLWE-PAKE	2020	[28]	3/3	C+S	$O(\frac{L'n}{\log n} + n \log^2 n)zb$	$O(L'n)zb$	$O(\frac{L'n}{\log n} + n \log n)M$	$O(\frac{L'n}{\log n} + n \log n)M$	0	ASPH
LWE-PAKE	2020	[28]	3/3	C+S	(nm)zb	$O(L'nm/\log q)zb$	$O(\frac{L'n}{\log n} + n \log n)M$	$O(\frac{L'n}{\log n} + n \log n)M$	0	ASPH
Erwig et al.	2020	[69]	3/3	None	(2n+1)fb+(2n)vb	(2n+1)fb+(4n)vb	(6n)G	(4n+1)G	(2n+2)G	RSS, OT
AucPace	2019	[90]	4/6	C+S	1fb+2vb	1fb+2vb	1G + 1κ	2G + 3κ	1G + 2κ	PBKDF, Map2Point
saPAKE-BB	2019	[58]	2/2	S	10fb+1vb	2fb+2vb	2G	2G + 2κ	1G + 1κ	CKEM, STOWF
Hwang et al.	2018	[9]	3/3	C	1fb+2vb	2fb+2vb	1G + 2κ	2G	1G + 1κ	Password-enc
Dupont et al.	2018	[70]	5/5	None	nfb+(2n+2)vb	(n)fb+(2n+2)vb	(4n+2)G	(4n+2)G	0	Garbled Circuits, OT
OPAQUE	2018	[7]	3/3	C+S	1fb+2vb+1mnb	1fb+1vb+1mnb	2G + 1κ	3G + 1κ	5G	Password-enc, OPRF
PFS-SPAKE2	2018	[14]	3/3	C+S	2fb+1vb	2fb+1vb	1G + 1κ	1G + 1κ	0	–
P-SPEKE	2018	[20]	1/2	None	2vb	2vb	1G	1G	0	Hash-to-group, KDF
VTBPEKE	2017	[49]	3/3	C	2fb+3vb	1fb+3vb	1G + 2κ	1G + 2κ	1G + 1κ	Symmetric enc
TBPEKE	2017	[49]	2/2	None	1fb+2vb	1fb+2vb	1G	1G	0	–
SESPAKE	2017	[96]	6/6	C+S	1fb+2vb	1fb+1vb	1G + 1κ	1G + 2κ	3G	MAC
Zhang-Yu	2017	[31]	2/2	None	$\omega(L'nm \log n)zb$	$\omega(L'nm \log n)zb$	(L'n)M	(L'n)M	0	SPHF, Peiker enc
Zhang et al.	2016	[27]	2/2	None	1fb+2vb+2mnb	1fb+2vb+1mnb	4G	4G	1G + 1κ	Password-enc, Sign
DE-PAKE	2016	[65]	5/6	C+S	1fb+4vb+1mnb	1fb+2vb+1mnb	3G + 1κ	7G + 1κ	6G	OPRF, HMQV
Abdalla et al.	2015	[115]	3/6	C+S	2fb+11vb+1mnb	2fb+11vb+1mnb	3G + 3κ	3G + 3κ	κ	NIZK
Patched SPEKE	2014	[88]	2/4	C+S	2vb	2vb	1G + 1κ	1G + 1κ	0	Hash-to-group
Canetti et al.	2012	[50]	5/5	C+S	(n)fb+(n)vb	(n)fb+(n)vb	(n)G + 1κ	(3n)G + 1κ	0	SPHF, PVW enc
Groce-Katz	2010	[56]	3/3	C	(2nm)zb	$O(L'nm)zb$	(1/2L'n)M	(1/2L'n)M	0	SPHF, Shoup enc
AugPAKE	2010	[101]	4/4	C+S	1fb+1mnb	1fb+1mnb	1G + 1κ	1G + 1κ	1G	–
Yang et al.	2010	[42]	3/3	C+S	1fb+15vb	1fb+12vb	11G	3G	4G	Password-enc, Sign
Yang et al.	2009	[41]	3/3	C+S	7fb+41vb+4mnb	2fb+53vb+2mnb	34G	2G	4G	Password-enc, Sign
Katz-Vaikuntanathan	2009	[68]	3/3	None	$\omega(L'nm \log n)zb$	$\omega(L'nm \log n)zb$	(L'n)M	(L'n)M	0	SPHF, Peikert enc
PACE v2	2008	[51]	4/5	C+S	2vb	2vb	1G + 2κ	1G + 1κ	0	Password-enc, Htc
SAE	2008	[98]	2/4	C+S	2vb	2vb	2G + 1κ	2G + 1κ	0	Hash-to-curve
J-PAKE	2008	[16]	3/6	C+S	2fb+11vb+1mnb	2fb+11vb+1mnb	3G + 3κ	3G + 3κ	κ	NIZK
SPAKE2+	2008	[94]	1/2	None	3fb+2vb	4fb+1vb	1G	1G	2κ	–
Wan et al.	2007	[33]	3/3n	C+S	2fb+(log n)vb	1fb+(n)vb	(log n)G + 3κ	(n)G + 1κ	(n)κ	Password-enc, Tree
PACE v1	2006	[83]	4/5	C+S	2vb	2vb	1G + 2κ	1G + 1κ	0	Password-enc, Htc
Abdalla-Pointcheval	2006	[107]	4/4n	C+S	(n+2)vb	(n+2)vb	4G + 1κ	4G + 1κ	0	Hash-to-group, Sign
Abdalla et al.	2005	[24]	3/5	C+S	2fb+1vb	2fb+1vb	2G + 1κ	2G + 1κ	2κ	PAKE, MAC
SPAKE2	2005	[36]	1/2	None	3fb+1vb	3fb+1vb	1G	1G	0	–
SPAKE1	2005	[36]	1/2	None	3fb+1vb	3fb+1vb	1G	1G	0	–
PAK-Z+	2005	[97]	3/3	C+S	1fb+1vb	1fb+1vb	1G + 1κ	1G + 3κ	1G + 2κ	Hash-to-group, Sign
Canetti et al.	2005	[105]	4/4	None	3vb+6mnb	2vb+4mnb	9G	7G	0	SPHF, CS enc
Jiang-Gong	2004	[93]	3/3	C+S	3fb+1vb	3fb+2vb	2G	1G + 1κ	0	Symmetric enc
PEKEP	2004	[35]	4/4	C+S	(m+1)vb	(m+1)vb	2G + 2κ	1G + 2κ	0	Password-enc
OKE	2003	[77]	3/3	C	1fb+1vb	1fb+1vb	1G + 1κ	1G	0	Password-enc
Gennaro-Lindell	2003	[40]	3/3	None	3vb+3mnb	3vb+3mnb	10G	9G	0	SPHF, CS enc
Kobara-Imai	2002	[92]	2/4	C+S	2fb+1vb	2fb+1vb	1G + 1κ	1G + 1κ	0	MAC
SRP-6	2002	[21]	6/6	C+S	2fb+1vb	1fb+1mnb	1G + 1κ	1G + 2κ	1G + 1κ	Mathematical func.
AMP	2001	[38]	4/4	C+S	1fb+1vb	2vb	1G + 1κ	1G + 1κ	1G + 1κ	–
KOY	2001	[91]	3/3	C	7fb+5vb+2mnb	7fb+5vb+2mnb	5G + 2κ	5G	0	Sign
PAK-RY	2001	[66]	3/3	C+S	2fb+2vb	1fb+3vb	1G + 2κ	1G + 1κ	1G	Hash-to-group, NIZK
PPK	2000	[34]	1/2	None	1fb+2vb	1fb+2vb	1G	1G	0	Hash-to-group
PAK	2000	[34]	3/3	C+S	1fb+2vb	1fb+2vb	1G + 1κ	1G + 1κ	0	Hash-to-group
AuthA	2000	[72]	2/2	S	1fb+1vb	1fb+1vb	1G + 1κ	1G + 1κ	1G	Password-enc
SNAPI	2000	[74]	4/4	C+S	1vb	1vb	2G + 2κ	1G + 2κ	0	Password-enc
SRP	1998	[10]	6/6	C+S	2fb+1vb	1fb+1mnb	1G + 1κ	1G + 3κ	1G + 1κ	Mathematical func.
OKE	1997	[73]	4/4	C+S	1vb	1vb	2G + 2κ	1G + 2κ	0	Password-enc
B-SPEKE	1997	[87]	3/5	C+S	3vb	1fb+3vb	1G + 1κ	2G + 1κ	2G	Hash-to-group
SPEKE	1996	[15]	3/4	C+S	2vb	2vb	1G + 1κ	1G + 1κ	0	Hash-to-group
M-EKE	1995	[76]	3/3	C+S	1fb+1vb	1fb+1vb	1G + 1κ	1G + 1κ	0	Password-enc
A-EKE	1993	[37]	5/5	C+S	1fb+1vb	1fb+1vb	1G + 2κ	1G + 2κ	0	Password-enc
EKE	1992	[13]	5/5	C+S	1fb+1vb	1fb+1vb	1G + 2κ	1G + 1κ	0	Password-enc

κ: κ-bit string
G: Group element
M: Matrix over lattice
ae: Authenticated encryption
zb: Multiplications in $\mathbb{Z}_q$
fb: Fixed-based exponentiation
vb: Variable-based exponentiation
mnb: Multi-exponentiation

sig: Signature
R/F: Round/Flow
pke: Public-key encryption
EA: Explicit authentication
OT: Oblivious transfer
Htg: Hash-to-group
Htc: Hash-to-curve
KEM: Key encapsulation mechanism

KDF: Key derivation function
SPHF: Smooth projective hash function
NIZK: Non-interactive zero-knowledge
OPRF: Oblivious pseudo-random function
ASPH: Approximate smooth projective hashing
PBKDF: Password-based key derivation function
CKEM: Conditional key encapsulation mechanism
LPKE: Lossy public key encryption

RSS: Robust secret sharing
MAC: Message authentication code
STOWF: Salted tight one-way function
Shoup enc: Shoup encryption [113]
Peiker enc: Peiker encryption [155]
PVW enc: Peikert et al. encryption [156]
CS enc: Cramer-Shoup encryption [109]
ECC: Error correcting codes

OPAQUE [7] and aEKE-HMQV [8]) will compute multi-exponentiation (mvb). The computation cost of 1vb (or 1mvb) is enough to complete authentication and key exchange (see the OKAPE-HMQV protocol [8]). But anonymous PAKE protocols (e.g., [27, 41, 42]) and multi-party PAKE protocols (e.g., [33, 107]) often require more computation, communication, and storage overhead. They strengthen the basic guarantees of security properties at the expense of efficiency. In addition, in 1999, Halevi and Krawczyk [145] pointed out that it is unavoidable to use public-key technology to resist offline guessing attacks. This is also reflected in Table 10: All PAKE protocols use public-key techniques, such as Diffie-Hellman key exchange and RSA.

Among all building blocks, the most commonly used cryptography tool (28/71=39.4%) is encryption, including password-based symmetric encryption, public-key encryption (e.g., Cramer-Shoup encryption [109] and Peiker encryption [155]). The second most used (18/71=25.3%) is hash-to-group/hash-to-curve. Other tools include Signature, Non-interactive zero-knowledge (NIZK), Oblivious pseudo-random function (OPRF), etc. In particular, OPRF has aroused widespread interest among researchers as an efficient cryptography tool. For example, the OPRF (i.e., 2HashDH [158]) used in OPAQUE [7] only requires one variable-based exponentiation per party. The comparison between OPAQUE [7] and its variant KHAPE [11] illustrates the advantages and disadvantages of using OPRF in a PAKE protocol. With OPRF, OPAQUE [7] is resilient to pre-computation attacks. By dispensing with the OPRF, KHAPE [11] can not resist pre-computation attacks, but it provides resilience to OPRF compromise and its performance has been improved (see Fig. 3).

We emphasize that inefficient PAKEs (e.g., [7, 71]) are not necessarily useless. Performance constraints and security goals for a PAKE vary depending on the expected application environment. For highly critical environments such as government offices, security is the first requirement. In order to achieve the desired level of security, it may be acceptable to use a slightly inefficient authentication protocol, such as OPAQUE [7]. Conversely, lower-risk environments may focus more on efficiency. A very useful but less secure authentication scheme may also be acceptable to service providers. With this in mind, it is wise and feasible to choose a specific PAKE for target environments and applications. After all, from a usability and security point of view, it is unlikely that any single scheme will suit all environments and target users.

6 DISCUSSION

Our review of password-authenticated key exchange (PAKE) protocols reveals a rich palette of useful ideas, and here we provide some insights.

The neglected part of PAKE. PAKEs ignore the password update phase for a long time, which is a disadvantage for its application. In other authentication schemes, such as the smart-card-based password authentication mechanism (e.g., [44]), the operations performed on the password during the password update phase will be described in detail, which also matches the practical situation. Conversely, most PAKEs do not pay attention to the update of the password. Remarkably, in a PAKE, the password cannot be updated locally. Achieving authentication inherently requires some common information (e.g., passwords) to be shared in advance. When the user updates a new password locally and the server still possesses the old password, the prerequisite for authentication is broken. In a word, clarifying how the password update phase works is not only closer to the real world, but also allows researchers to have a broader understanding of the design of PAKEs.

For better comprehension, we propose a general framework (as shown in Fig. 4) for password update in PAKE protocols, intended to provide a common reference point for systematic analysis and comparison. In the symmetric PAKEs [13, 148], aPAKEs [8, 147], and saPAKEs [7, 58], the most direct approach is to authenticate with a standard login and then perform a complete re-registration. Consequently, the round complexity and computational cost of the update phase are bounded by

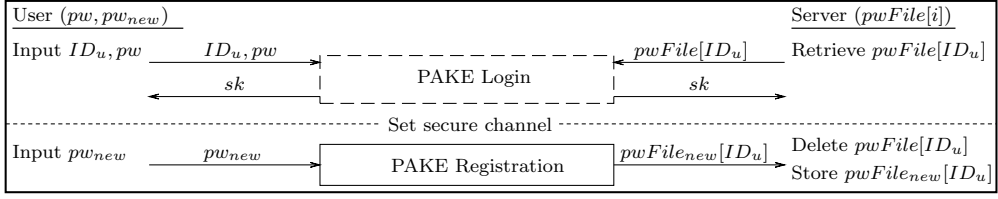


Fig. 4. A generic framework for the password update phase of PAKE. ID_u denotes the identity of user U . pw and pw_{new} denote the original and new passwords, respectively. $pwFile[i]$ denotes the password file stored on the server S , and sk is the session key. The [dashed box] indicates operations over the public channel, whereas the [solid box] indicates operations the secure channel. U can establish a secure channel with S via PAKE using pw over the public channel, and then generates a new password file $pwFile_{new}[ID_u]$ for S .

the sum of these two steps. While dedicated constructions can reduce this overhead, the baseline strategy remains broadly applicable and reliably achieves secure password rotation.

Further work on the online password update without revealing the user's plain-text password to the server is needed, considering the disturbing fact that the user's plain-text password may be unconsciously logged, unauthorizedly accessed, and even misused by insider attackers. See a concrete catastrophic example from Twitter [159].

The dilemma of deploying PAKE. The deployment of PAKE has two basic requirements: 1) a secure channel is required during the registration phase, and 2) an user must be modified to support and execute a PAKE [22]. The fundamental assumption of PAKE is that both parties pre-share a password. It means that users either share a password face-to-face or send the password under the protection of a secure channel such as TLS. The deployment of PAKE on the user end usually requires a browser plugin or a software application [14, 65]. Currently, the widely accepted password authentication scheme (i.e., password-over-TLS) does not require any modifications to the client. One possible approach is to dynamically (and securely) load the PAKE execution code to the client when the user requests authentication. Then, the user enters the password and executes the PAKE with the server. But ensuring secure dynamic code loading is challenging [160].

Future trends of PAKE. Based on the above analysis of existing PAKEs and considering the development trends of computing and communication techniques, we identify four potential directions for the future design of PAKEs.

- *Anonymous PAKE protocol.* With more and more sensitive services and applications (that involve personal information) made online, how to protect user privacy is becoming a major concern. For instance, the 2023 Verizon Data Breach Investigations Report (DBIR) [55] reveals that about 53% of data breaches include personally identifiable information (PII) such as email, name, user ID, and mobile phone number. If these PII data are obtained by malicious network service providers rather than attackers, it is equally disturbing (e.g., activity tracking and targeted advertising). Therefore, it is desirable to take measures to protect users' identities and associated activities from unnecessary exposure. However, password-over-TLS does not implement anonymity. Because the server authenticates the user's (user ID, password)-pair. In contrast, the anonymous PAKE can achieve authentication and anonymity protection simultaneously, with user ID not only hidden from the adversary but also from the server.
- *Blockchain-based PAKE protocol.* Due to promising properties (e.g., decentralization and non-repudiation), researchers have attempted to apply blockchain to various protocols, e.g., blockchain-based self-tallying voting protocols [161, 162] and privacy-preserving payment protocols [163, 164]. In addition to being applied to distributed authentication, blockchain can also be used to design anonymous authentication protocols. The latest research [165] proposes a distributed authentication protocol for the Internet of Things based on a public

blockchain. Fully executing PAKEs on-chain may introduce latency and privacy challenges. Therefore, the focus of blockchain-based PAKE designs is to leverage blockchain as a verifiable and tamper-resistant auxiliary component in password authentication systems. For example, blockchain can be used to enhance auditability [166] or record cryptographic commitments in a verifiable manner [167]. Still, how to incorporate blockchain as an auxiliary component into password authentication protocols remains an interesting and promising line of research.

- *Lattice-based PAKE protocol.* This type of protocol currently receives growing attention. With the growing threat posed by quantum computers to classical hardness assumptions (e.g., discrete logarithm and integer factorization problems), PAKE protocols based on classical hardness assumptions are facing increasing security risks. Quantum adversaries can efficiently break PAKEs (e.g., OPAQUE [7] and KHAPE [11]) using the Shor algorithm [62], and can apply the Grover algorithm [63] to accelerate brute-force search and hash collision attacks. As a result, quantum-resistant PAKE protocols have emerged as an important subclass of PAKE. Research on quantum-resistant PAKE protocols has become a hot topic [168]. Among all post-quantum algorithms (e.g., hash-based, code-based, lattice-based, and isogeny-based), lattice-based schemes appear to be the most promising general-purpose algorithms due to their provable security and relatively low computation cost [169]. Notably, three of the four post-quantum encryption algorithms standardized by NIST are lattice-based [170]. Designing secure and usable lattice-based PAKE protocols remains a trend.
- *Identity-based PAKE protocol.* As mentioned above, (symmetric) PAKE may be more suitable in symmetric settings. But if two parties share a plaintext password, the password is immediately exposed after one party is compromised. At CRYPTO'22, Cremers et al. [71] proposed the CRISP protocol combining symmetric PAKE with identity-based authentication, namely symmetric identity-based PAKE. CRISP is resistant to pre-computation attacks, which is the highest level of security that asymmetric PAKE (including aPAKE like OKAPE-HMQV [8] and saPAKE like OPAQUE [7]) can provide. Compared to non-identity based PAKE (particularly strong asymmetric PAKE, i.e., saPAKE), one critical defect of identity-based PAKE is that both parties need to store a password file. Thus, users can only log in on the device with this password file. In addition, identity-based PAKE still requires more exploration in terms of efficiency. In all, identity-based PAKE protocols are likely to be a promising direction.

7 CONCLUSION

We have provided an extensive review of the most notable works to date on password-authenticated key exchange (PAKE) protocols. We propose a list of 13 desirable properties, allowing PAKE protocols to be rated across a common spectrum for a fair evaluation. We categorize existing PAKE protocols into seven types according to their underlying design strategies, and investigate the pros and cons of each type. We further classify them from five other key perspectives (i.e., symmetry, number of participants, hardness assumptions, security goals, and round complexity), and conduct a large-scale comparative evaluation of 71 PAKE protocols. We have also pointed out four promising future research directions. We believe this work provides a thorough and in-depth understanding of how to design PAKE protocols that offer acceptable trade-offs between security and usability.

REFERENCES

- [1] L. O’Gorman, Comparing passwords, tokens, and biometrics for user authentication, IEEE 91 (12) (2003) 2021–2040.
- [2] J. Bonneau, C. Herley, P. C. van Oorschot, F. Stajano, The quest to replace passwords: A framework for comparative evaluation of web authentication schemes, in: Proc. IEEE S&P 2012, pp. 553–567.
- [3] J. Bonneau, C. Herley, P. C. van Oorschot, F. Stajano, Passwords and the evolution of imperfect authentication, Commun. ACM 58 (7) (2015) 78–87.

- [4] V. Zimmermann, From the quest to replace passwords towards support- ing secure and usable password creation, Ph.D. thesis, Technical University of Darmstadt, Germany (2021).
- [5] L. H. Newman, Why the Password Isn't Dead Quite Yet, <https://bit.ly/46xY886> (June 2021).
- [6] The Password Isn't Dead Yet. You Need a Hardware Key, <https://bit.ly/43bXo5K> (Apr. 2023).
- [7] S. Jarecki, H. Krawczyk, J. Xu, OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks, in: Proc. EUROCRYPT 2018, pp. 456–486.
- [8] B. F. D. Santos, Y. Gu, S. Jarecki, H. Krawczyk, Asymmetric PAKE with low computation and communication, in: Proc. EUROCRYPT 2022, pp. 127–156.
- [9] J. Y. Hwang, S. Jarecki, T. Kwon, J. Lee, J. S. Shin, J. Xu, Round-reduced modular construction of asymmetric password-authenticated key exchange, in: Proc. SCN 2018, pp. 485–504.
- [10] T. D. Wu, The secure remote password protocol, in: Proc. NDSS 1998, pp. 97–111.
- [11] Y. Gu, S. Jarecki, H. Krawczyk, KHAPE: Asymmetric PAKE from key-hiding key exchange, in: Proc. CRYPTO 2021.
- [12] J. S. Shin, M. Jo, J. Y. Hwang, J. Lee, A verifier-based password-authenticated key exchange using tamper-proof hardware, The Computer Journal 64 (8) (2021) 1293–1302.
- [13] S. M. Bellovin, M. Merritt, Encrypted key exchange: Password-based protocols secure against dictionary attacks, in: Proc. IEEE S&P 1992, pp. 72–84.
- [14] J. Becerra, D. Ostrev, M. Skrobot, Forward secrecy of SPAKE2, in: Proc. ProvSec 2018, pp. 366–384.
- [15] D. P. Jablon, Strong password-only authenticated key exchange, Comput. Commun. Rev. 26 (5) (1996) 5–26.
- [16] F. Hao, P. Y. A. Ryan, Password authenticated key exchange by juggling, in: Proc. SPW 2008, pp. 159–171.
- [17] R. Fillion, Secure Remote Password (SRP): How 1Password uses it, <https://bit.ly/3roH1FW> (Feb. 2018).
- [18] Apple, HomeKitADK, <https://github.com/apple/HomeKitADK> (Oct. 2021).
- [19] Firefox Sync, <https://www.mozilla.org/en-US/firefox/sync/> (June 2023).
- [20] F. Hao, R. Metere, S. F. Shahandashti, C. Dong, Analysing and patching SPEKE in ISO/IEC, IEEE Trans. Inf. Forensics Secur. 13 (11) (2018) 2844–2855.
- [21] T. D. Wu, SRP-6: Improvements and refinements to the secure remote password protocol, Submission to IEEE P1363.2 (Oct. 2002).
- [22] F. Hao, P. C. van Oorschot, Sok: Password-authenticated key exchange - theory, practice, standardization and real-world lessons, in: Proc. AsiaCCS 2022, pp. 697–711.
- [23] S. Patel, Number theoretic attacks on secure password schemes, in: Proc. IEEE S&P 1997, pp. 236–247.
- [24] M. Abdalla, P. Fouque, D. Pointcheval, Password-based authenticated key exchange in the three-party setting, in: Proc. PKC 2005, pp. 65–84.
- [25] J. Katz, P. D. MacKenzie, G. Taban, V. D. Gligor, Two-server password-only authenticated key exchange, in: Proc. ACNS 2005, pp. 1–16.
- [26] M. I. G. Vasco, A. L. P. del Pozo, C. Soriente, A key for john doe: Modeling and designing anonymous password-authenticated key exchange protocols, IEEE Trans. Dependable Secur. Comput. 18 (3) (2021) 1336–1353.
- [27] Z. Zhang, K. Yang, X. Hu, Y. Wang, Practical anonymous password authentication and TLS with anonymous client authentication, in: Proc. ACM CCS 2016, pp. 1179–1191.
- [28] S. Jiang, G. Gong, J. He, K. Nguyen, H. Wang, Pakes: New framework, new techniques and more efficient lattice-based constructions in the standard model, in: Proc. PKC 2020, pp. 396–427.
- [29] P. Ren, X. Gu, Practical post-quantum password-authenticated key exchange based-on module-lattice, in: Proc. ICISC 2021, pp. 137–156.
- [30] J. Katz, V. Vaikuntanathan, Round-optimal password-based authenticated key exchange, in: Proc. TCC 2011.
- [31] J. Zhang, Y. Yu, Two-round PAKE from approximate SPH and instantiations from lattices, in: Proc. ASIACRYPT 2017.
- [32] Z. Li, D. Wang, E. Morais, Quantum-safe round-optimal password authentication for mobile devices, IEEE Trans. Dependable Secur. Comput. 19 (3) (2022) 1885–1899.
- [33] Z. Wan, R. H. Deng, F. Bao, B. Preneel, n PAKE⁺: A hierarchical group password-authenticated key exchange protocol using different passwords, in: Proc. ICICS 2007, pp. 31–43.
- [34] V. Boyko, P. D. MacKenzie, S. Patel, Provably secure password-authenticated key exchange using Diffie-Hellman, in: Proc. EUROCRYPT 2000, pp. 156–171.
- [35] M. Zhang, New approaches to password authenticated key exchange based on RSA, in: Proc. ASIACRYPT 2004.
- [36] M. Abdalla, D. Pointcheval, Simple password-based encrypted key exchange protocols, in: Proc. CT-RSA 2005.
- [37] S. M. Bellovin, M. Merritt, Augmented encrypted key exchange: A password-based protocol secure against dictionary attacks and password file compromise, in: Proc. ACM CCS 1993, pp. 244–250.
- [38] T. Kwon, Authentication and key agreement via memorable passwords, in: Proc. NDSS 2001, pp. 1–21.
- [39] H. Krawczyk, HMQV: A high-performance secure diffie-hellman protocol, in: Proc. CRYPTO 2005, pp. 546–566.
- [40] R. Gennaro, Y. Lindell, A framework for password-based authenticated key exchange, in: Proc. EUROCRYPT 2003.
- [41] Y. Yang, J. Zhou, J. Weng, F. Bao, A new approach for anonymous password authentication, in: Proc. ACSAC 2009.

- [42] Y. Yang, J. Zhou, J. W. Wong, F. Bao, Towards practical anonymous password authentication, in: Proc. ACSAC 2010.
- [43] M. Bellare, D. Pointcheval, P. Rogaway, Authenticated key exchange secure against dictionary attacks, in: Proc. EUROCRYPT 2000, pp. 139–155.
- [44] D. Wang, P. Wang, Two birds with one stone: Two-factor authentication with security beyond conventional bound, IEEE Trans. Depend. Secur. Comput. 15 (4) (2018) 708–722.
- [45] D. Wang, P. Wang, On the implications of Zipf's law in passwords, in: Proc. ESORICS 2016, pp. 111–131.
- [46] D. Wang, D. He, P. Wang, C. Chu, Anonymous two-factor authentication in distributed systems: Certain goals are beyond attainment, IEEE Trans. Depend. Secur. Comput. 12 (4) (2015) 428–442.
- [47] D. Wang, Z. Zhang, P. Wang, Targeted online password guessing: An underestimated threat, in: Proc. ACM CCS 16.
- [48] D. Wang, H. Cheng, P. Wang, X. Huang, G. Jian, Zipf's law in passwords, IEEE Trans. Inform. Foren. Secur. 12 (11) (2017) 2776–2791.
- [49] D. Pointcheval, G. Wang, VTBPEKE: Verifier-based two-basis password exponential key exchange, in: Proc. AsiaCCS 2017, pp. 301–312.
- [50] R. Canetti, D. Dachman-Soled, V. Vaikuntanathan, H. Wee, Efficient password authenticated key exchange via oblivious transfer, in: Proc. PKC 2012, pp. 449–466.
- [51] J.-S. Coron, A. Gouget, T. Icart, P. Paillier, Supplemental access control (PACE v2): security analysis of PACE integrated mapping, Cryptography and Security (2012) 207–232.
- [52] J. Bonneau, The science of guessing: Analyzing an anonymized corpus of 70 million passwords, in: Proc. IEEE S&P 2012, pp. 538–552.
- [53] Z. Chen, S. Pu, S. Zhu, Traceback technology for anonymous network, Journal of Computer Research and Development 49 (S2) (2012) 111–117.
- [54] P. Mayer, N. Gerber, B. Reinheimer, P. Rack, K. Braun, M. Volkamer, I (don't) see what you typed there! shoulder-surfing resistant password entry on gamepads, in: Proc. CHI 2019, pp. 1–12.
- [55] Verizon, 2023 Data Breach Investigations Report, <https://www.verizon.com/business/resources/reports/dbir/> (July 2023).
- [56] A. Groce, J. Katz, A new framework for efficient password-based authenticated key exchange, in: Proc. ACM CCS 2010, pp. 516–525.
- [57] J. Katz, R. Ostrovsky, M. Yung, Forward secrecy in password-only key exchange protocols, in: Proc. SCN 2002.
- [58] T. Bradley, S. Jarecki, J. Xu, Strong asymmetric PAKE based on trapdoor CKEM, in: Proc. CRYPTO 2019, pp. 798–825.
- [59] T. Y. Youn, Y. H. Park, C. H. Kim, J. Lim, Weakness in a rsa-based password authenticated key exchange protocol, Inf. Process. Lett. 108 (6) (2008) 339–342.
- [60] B. A. LaMacchia, K. E. Lauter, A. Mityagin, Stronger security of authenticated key exchange, in: Proc. ProvSec 2007.
- [61] J. Preskill, Quantum computing in the nisyq era and beyond, Quantum 2 (2018) 79.
- [62] P. W. Shor, Algorithms for quantum computation: discrete logarithms and factoring, in: Proc. FOCS 1994, pp. 124–134.
- [63] L. K. Grover, A fast quantum mechanical algorithm for database search, in: Proc. STOC 1996, pp. 212–219.
- [64] E. Bresson, O. Chevassut, D. Pointcheval, New security results on encrypted key exchange, in: Proc. PKC 2004.
- [65] S. Jarecki, H. Krawczyk, M. Shirvanian, N. Saxena, Device-enhanced password protocols with optimal online-offline protection, in: Proc. AsiaCCS 2016, pp. 177–188.
- [66] P. D. MacKenzie, More efficient password-authenticated key exchange, in: Proc. CT-RSA 2001, pp. 361–377.
- [67] P. MacKenzie, The PAK suite: Protocols for password-authenticated key exchange, Contribution to IEEE P1363 (Dec. 2002).
- [68] J. Katz, V. Vaikuntanathan, Smooth projective hashing and password-based authenticated key exchange from lattices, in: Proc. ASIACRYPT 2009, pp. 636–652.
- [69] A. Erwig, J. Hesse, M. Orlt, S. Riahi, Fuzzy asymmetric password-authenticated key exchange, in: Proc. ASIACRYPT 2020, pp. 761–784.
- [70] P. Dupont, J. Hesse, D. Pointcheval, L. Reyzin, S. Yakoubov, Fuzzy password-authenticated key exchange, in: Proc. EUROCRYPT 2018, pp. 393–424.
- [71] C. Cremers, M. Naor, S. Paz, E. Ronen, CHIP and CRISP: protecting all parties against compromise through identity-binding pakes, in: Proc. CRYPTO 2022, pp. 668–698.
- [72] M. Bellare, P. Rogaway, The AuthA protocol for password-based authenticated key exchange, Contribution to IEEE P1363 (Mar. 2000).
- [73] S. Lucks, Open key exchange: How to defeat dictionary attacks without encrypting public keys, in: Proc. SPW 1997.
- [74] P. D. MacKenzie, S. Patel, R. Swaminathan, Password-authenticated key exchange based on RSA, in: Proc. ASIACRYPT 2000, pp. 599–613.
- [75] IEEE standard specification for password-based public-key cryptographic techniques, https://standards.ieee.org/standard/1363_2-2008.html (Jane 2009).

- [76] M. Steiner, G. Tsudik, M. Waidner, Refinement and extension of encrypted key exchange, *ACM SIGOPS Oper. Syst. Rev.* 29 (3) (1995) 22–30.
- [77] E. Bresson, O. Chevassut, D. Pointcheval, Security proofs for an efficient password-based key exchange, in: *Proc. ACM CCS 2003*, pp. 241–250.
- [78] G. F. O. for Information Security, Advanced security mechanisms for machine readable travel documents part 1 – emrtds with bac/pacev2 and eacv1, <https://bit.ly/46BSUbx> (Mar. 2012).
- [79] C. Gentry, P. D. MacKenzie, Z. Ramzan, A method for making password-based key exchange resilient to server compromise, in: *Proc. CRYPTO 2000*, pp. 142–159.
- [80] M. Barbosa, K. Gellert, J. Hesse, S. Jarecki, Bare pake: universally composable key exchange from just passwords, in: *Proc. CRYPTO 2024*, pp. 183–217.
- [81] A. Arriaga, P. Y. Ryan, M. Skrobot, Sweetpake: Key exchange with decoy passwords, in: *Proc. AsiaCCS 2024*, pp. 1017–1033.
- [82] J. Bootle, S. Faller, J. Hesse, K. Hostáková, J. Ottenhues, Generalized fuzzy password-authenticated key exchange from error correcting codes, in: *Proc. ASIACRYPT 2023*, pp. 110–142.
- [83] J. Bender, M. Fischlin, D. Kügler, Security analysis of the PACE key-agreement protocol, in: *Proc. ISC 2009*, pp. 33–48.
- [84] F. Wei, C. Ma, Q. Cheng, Gateway-oriented password-authenticated key exchange based on RSA, *Chinese Journal of Computers.* 34 (1) (2011) 38–46.
- [85] D. Wang, P. Wang, M. Li, Cryptanalysis and improvement of gateway-oriented password-authenticated key exchange based on RSA, *Acta Electronica Sinica.* 43 (1) (2015) 176–184.
- [86] W. Diffie, M. E. Hellman, New directions in cryptography, *IEEE Trans. Inf. Theory* 22 (6) (1976) 644–654.
- [87] D. P. Jablon, Extended password key exchange protocols immune to dictionary attacks, in: *Proc. WETICE 1997*.
- [88] F. Hao, S. F. Shahandashti, The SPEKE protocol revisited, in: *Proc. SSR 2014*, pp. 26–38.
- [89] Information technology — Security techniques — Key management — Part 4: Mechanisms based on weak secrets, <https://www.iso.org/standard/67933.html> (Nov. 2017).
- [90] B. Haase, B. Labrique, Aucpace: Efficient verifier-based PAKE protocol tailored for the iiot, *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2019 (2) (2019) 1–48.
- [91] J. Katz, R. Ostrovsky, M. Yung, Efficient password-authenticated key exchange using human-memorable passwords, in: *Proc. EUROCRYPT 2001*, pp. 475–494.
- [92] K. Kobara, H. Imai, Pretty-simple password-authenticated key-exchange protocol proven to be secure in the standard model, *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.* 85-A (10) (2002) 2229–2237.
- [93] S. Jiang, G. Gong, Password based key exchange with mutual authentication, in: *Proc. SAC 2004*, pp. 267–279.
- [94] D. Cash, E. Kiltz, V. Shoup, The twin diffie-hellman problem and applications, in: *Proc. EUROCRYPT 2008*, pp. 127–145.
- [95] V. Shoup, Security analysis of itspake2+, in: *Proc. TCC 2020*, pp. 31–60.
- [96] S. Smyshlyaev, E. K. Alekseev, I. B. Oshkin, V. Popov, The security evaluated standardized password-authenticated key exchange (SESPAKEY) protocol, *RFC 8133* (2017) 1–51.
- [97] C. Gentry, P. MacKenzie, Z. Ramzan, PAK-Z+, Submission to IEEE P1363.2 (Aug. 2005).
- [98] D. Harkins, Simultaneous authentication of equals: A secure, password-based key exchange for mesh networks, in: *Proc. SensorComm 2008*, p. 839–844.
- [99] M. Qi, W. Hu, Y. Tai, Sae+: One-round provably secure asymmetric sae protocol for client-server model, *IEEE Trans. Inf. Forensics Secur.* 19 (2024) 3906 – 3913.
- [100] I. McQuoid, M. Rosulek, L. Roy, Minimal symmetric PAKE and 1-out-of-n OT from programmable-once public functions, in: *Proc. ACM CCS 2020*, pp. 425–442.
- [101] S. Shin, K. Kobara, Efficient augmented password-only authentication and key exchange for ikev2, *RFC 6628* (2012).
- [102] F. Hao, J-PAKE: password-authenticated key exchange by juggling, *RFC 8236* (2017) 1–15.
- [103] OpenSSL Project, <https://www.openssl.org> (June 2023).
- [104] Pale Moon Sync, <https://www.palemoon.org/sync/> (June 2023).
- [105] R. Canetti, S. Halevi, J. Katz, Y. Lindell, P. D. MacKenzie, Universally composable password-based key exchange, in: *Proc. EUROCRYPT 2005*, pp. 404–421.
- [106] D. Fiore, R. Gennaro, Identity-based key exchange protocols without pairings, *Trans. Comput. Sci.* 10 (2010) 42–77.
- [107] M. Abdalla, D. Pointcheval, A scalable password-based group key exchange protocol in the standard model, in: *Proc. ASIACRYPT 2006*, pp. 332–347.
- [108] D. Micciancio, C. Peikert, Trapdoors for lattices: Simpler, tighter, faster, smaller, in: *Proc. EUROCRYPT 2012*.
- [109] R. Cramer, V. Shoup, Universal hash proofs and a paradigm for adaptive chosen ciphertext secure public-key encryption, in: *Proc. EUROCRYPT 2002*, pp. 45–64.
- [110] C. Peikert, Lattice cryptography for the internet, in: *Proc. PQCrypto 2014*, pp. 197–219.
- [111] M. Naor, M. Yung, Universal one-way hash functions and their cryptographic applications, in: *Proc. ACM STOC 1989*.
- [112] K. Kurosawa, Y. Desmedt, A new paradigm of hybrid encryption scheme, in: *Proc. CRYPTO 2004*, pp. 426–442.

- [113] V. Shoup, A proposal for an ISO standard for public key encryption, *IACR Cryptol. ePrint Arch.* (2001) 112.
- [114] J. Hesse, Separating symmetric and asymmetric password-authenticated key exchange, in: *Proc. SCN 2020*.
- [115] M. Abdalla, F. Benhamouda, P. MacKenzie, Security of the J-PAKE password-authenticated key exchange protocol, in: *Proc. IEEE S&P 2015*, pp. 571–587.
- [116] R. Lu, Z. Cao, Simple three-party key exchange protocol, *Comput. Secur.* 26 (1) (2007) 94–97.
- [117] H. Huang, A simple three-party password-based key exchange protocol, *Int. J. Commun. Syst.* 22 (7) (2009) 857–862.
- [118] E. Yoon, K. Yoo, Cryptanalysis of a simple three-party password-based key exchange protocol, *Int. J. Commun. Syst.* 24 (4) (2011) 532–542.
- [119] S. Tallapally, Security enhancement on simple three party PAKE protocol, *Inf. Technol. Control.* 41 (1) (2012) 15–22.
- [120] M. S. Farash, M. Ahmadian-Attari, An enhanced and secure three-party password-based authenticated key exchange protocol without using server's public-keys and symmetric cryptosystems, *Inf. Technol. Control.* 43 (2) (2014) 143–150.
- [121] Y. Lu, L. Li, H. Peng, Y. Yang, A three-party password-based authenticated key exchange protocol for wireless communications, *Inf. Technol. Control.* 44 (4) (2015) 404–409.
- [122] C. Chen, K. Wang, K. Yeh, B. Xiang, T. Wu, Attacks and solutions on a three-party password-based authenticated key exchange protocol for wireless communications, *J. Ambient Intell. Humaniz. Comput.* 10 (8) (2019) 3133–3142.
- [123] H. Xiong, Y. Wu, Z. Lu, A survey of group key agreement protocols with constant rounds, *ACM Comput. Surv.* 52 (3) (2019) 57:1–57:32.
- [124] V. S. Miller, Use of elliptic curves in cryptography, in: *Proc. CRYPTO 1985*, pp. 417–426.
- [125] M. Kobitz, Elliptic curve cryptosystems, *Mathematics of Computation* 48 (177) (1987) 203–209.
- [126] T. ElGamal, A public key cryptosystem and a signature scheme based on discrete logarithms, *IEEE Trans. Inf. Theory* 31 (4) (1985) 469–472.
- [127] E. Barker, NIST special publication 800-57 part 1 revision 5, <https://doi.org/10.6028/NIST.SP.800-57pt1r5> (May 2020).
- [128] D. Park, C. Boyd, S. Moon, Forward secrecy and its application to future mobile communications security, in: *Proc. PKC 2000*, pp. 433–445.
- [129] R. L. Rivest, A. Shamir, L. M. Adleman, A method for obtaining digital signatures and public-key cryptosystems, *Commun. ACM* 21 (2) (1978) 120–126.
- [130] O. Regev, On lattices, learning with errors, random linear codes, and cryptography, in: *Proc. STOC 2005*, pp. 84–93.
- [131] D. Sikeridis, P. Kampanakis, M. Devetsikiotis, Post-quantum authentication in TLS 1.3: A performance study, in: *Proc. NDSS 2020*, pp. 1–16.
- [132] W. Diffie, P. C. van Oorschot, M. J. Wiener, Authentication and authenticated key exchanges, *Des. Codes Cryptogr.* 2 (2) (1992) 107–125.
- [133] C. Ma, D. Wang, S. Zhao, Security flaws in two improved remote user authentication schemes using smart cards, *Int. J. Commun. Syst.* 27 (10) (2014) 2215–2227.
- [134] E. Rescorla, The transport layer security protocol version 1.3, <https://www.rfc-editor.org/rfc/rfc8446.txt> (Aug. 2018).
- [135] D. Wang, W. Li, P. Wang, Measuring two-factor authentication schemes for real-time data access in industrial wireless sensor networks, *IEEE Trans. Ind. Inform.* 14 (8) (2018) 4081–4092.
- [136] A. T. Sherman, E. Lanus, M. Liskov, E. Ziegler, R. Chang, E. Golaszewski, R. Wnuk-Fink, C. J. Bonyadi, M. Yaksetig, I. Blumenfeld, Formal methods analysis of the secure remote password protocol, in: *Proc. Logic, Language, and Security 2020*, pp. 103–126.
- [137] Y. He, F. Sun, Q. Li, J. He, L. Wang, A survey on public key mechanism in wireless sensor networks, *Chinese Journal of Computers* 381–408.
- [138] J. Don, S. Fehr, C. Majenz, C. Schaffner, Security of the fiat-shamir transformation in the quantum random-oracle model, in: *Proc. CRYPTO 2019*, pp. 356–383.
- [139] G. Alagic, C. Majenz, A. Russell, F. Song, Quantum-access-secure message authentication via blind-unforgeability, in: *Proc. EUROCRYPT 2020*, pp. 788–817.
- [140] J. Coron, J. Patarin, Y. Seurin, The random oracle model and the ideal cipher model are equivalent, in: *Proc. CRYPTO 2008*, pp. 1–20.
- [141] R. Canetti, Universally composable security: A new paradigm for cryptographic protocols, in: *Proc. FOCS 2001*.
- [142] M. Abdalla, M. Barbosa, T. Bradley, S. Jarecki, J. Katz, J. Xu, Universally composable relaxed password authenticated key exchange, in: *Proc. CRYPTO 2020*, pp. 278–307.
- [143] S. Shin, K. Kobara, H. Imai, Very-efficient anonymous password-authenticated key exchange and its extensions, in: *Proc. AAECC 2009*, pp. 149–158.
- [144] X. Hu, J. Zhang, Z. Zhang, J. Xu, Universally composable anonymous password authenticated key exchange, *Sci. China Inf. Sci.* 60 (5) (2017) 52107.
- [145] S. Halevi, H. Krawczyk, Public-key cryptography and password protocols, *ACM Trans. Inf. Syst. Secur.* 2 (3) (1999) 230–268.

- [146] D. Wang, P. Wang, On the anonymity of two-factor authentication schemes for wireless sensor networks: Attacks, principle and solutions, *Comput. Netw.* 73 (C) (2014) 41–57.
- [147] Y. Lyu, S. Liu, S. Han, Efficient asymmetric pake compiler from kem and ae, in: *Proc. ASIACRYPT 2025*, pp. 34–65.
- [148] Y. Lyu, S. Liu, S. Han, Universal composable password authenticated key exchange for the post-quantum world, in: *Proc. EUROCRYPT 2024*, pp. 120–150.
- [149] S. Gatlan, 533 million Facebook users' phone numbers and personal data have been leaked online, <https://www.businessinsider.com/stolen-data-of-533-million-facebook-users-leaked-online-2021-4> (Apr. 2021).
- [150] A. Holmes, Twitter data leak – over 200 million user data open to the public, <https://cybernews.com/news/235-million-twitter-user-data-leak/> (Jan. 2023).
- [151] C. Team, Scraped data of 500 million LinkedIn users being sold online, 2 million records leaked as proof, <https://cybernews.com/news/stolen-data-of-500-million-linkedin-users-being-sold-online-2-million-leaked-as-proof-2/> (Oct. 2022).
- [152] A. Menezes, M. Qu, S. Vanstone, Some new key agreement protocols providing mutual implicit authentication (1995).
- [153] W. Chen, D. Ding, H. Dong, G. Wei, Distributed resilient filtering for power systems subject to denial-of-service attacks, *IEEE Trans. Syst. Man Cybern. Syst.* 49 (8) (2019) 1688–1697.
- [154] S. Hu, D. Yue, Q. Han, X. Xie, X. Chen, C. Dou, Observer-based event-triggered control for networked linear systems subject to denial-of-service attacks, *IEEE Trans. Cybern.* 50 (5) (2020) 1952–1964.
- [155] C. Peikert, Public-key cryptosystems from the worst-case shortest vector problem: extended abstract, in: *Proc. STOC 2009*, pp. 333–342.
- [156] C. Peikert, V. Vaikuntanathan, B. Waters, A framework for efficient and composable oblivious transfer, in: *Proc. CRYPTO 2008*, pp. 554–571.
- [157] J. Katz, R. Ostrovsky, M. Yung, Efficient and secure authenticated key exchange using weak passwords, *J. ACM* 57 (1) (2009) 3:1–3:39.
- [158] S. Jarecki, A. Kiayias, H. Krawczyk, Round-optimal password-protected secret sharing and T-PAKE in the password-only model, in: *Proc. ASIACRYPT 2014*, pp. 233–253.
- [159] C. Gartenberg, Twitter advising all 330 million users to change passwords after bug exposed them in plain text, <https://bit.ly/3O8wHuE> (May 2018).
- [160] O. Or-Meir, N. Nissim, Y. Elovici, L. Rokach, Dynamic malware analysis in the modern era - A state of the art survey, *ACM Comput. Surv.* 52 (5) (2019) 88:1–88:48.
- [161] Y. Li, W. Susilo, G. Yang, Y. Yu, D. Liu, X. Du, M. Guizani, A blockchain-based self-tallying voting protocol in decentralized iot, *IEEE Trans. Dependable Secur. Comput.* 19 (1) (2022) 119–130.
- [162] W. Xue, Y. Yang, Y. Li, H. Pang, R. H. Deng, Acb-vote: Efficient, flexible, and privacy-preserving blockchain-based score voting with anonymously convertible ballots, *IEEE Trans. Inf. Forensics Secur.* 18 (2023) 3720–3734.
- [163] M. F. Esgin, R. Steinfeld, R. K. Zhao, Matrict^+ : More efficient post-quantum private blockchain payments, in: *Proc. IEEE S&P 2022*, pp. 1281–1298.
- [164] S. Dziembowski, L. Eeckey, S. Faust, D. Malinowski, Perun: Virtual payment hubs over cryptocurrencies, in: *Proc. IEEE S&P 2019*, pp. 106–123.
- [165] G. Yuan, S. Wen, Q. Tang, Y. Ma, S. Feng, Distributed authentication method of internet of things based on public blockchain, *Computer Engineering and Design* 51 (5) (2021) 1859–1866.
- [166] A. Roscoe, P. Y. Ryan, Auditable pakes: approaching fair exchange without a ttp, in: *Proc. SPW 2017*, pp. 278–297.
- [167] P. Szalachowski, Password-authenticated decentralized identities, *IEEE Transactions on Information Forensics and Security* 16 (2021) 4801–4810.
- [168] N. Alnahawi, D. Haas, E. Mauß, A. Wiesmaier, SoK: PQC PAKEs - cryptographic primitives, design and security, in: *Cryptology ePrint Archive*, Paper 2025/119, <https://eprint.iacr.org/2025/119>.
- [169] Q. Wang, D. Wang, C. Cheng, D. He, Quantum2fa: Efficient quantum-resistant two-factor authentication scheme for mobile devices, *IEEE Trans. Dependable Secur. Comput.* 20 (1) (2023) 193–208.
- [170] NIST, PQC Standardization Process: Announcing Four Candidates to be Standardized, Plus Fourth Round Candidates, <https://bit.ly/3DnrNUP> (July 2022).

Received 1 November 2023; revised 1 November 2023; accepted 1 November 2023